

Rudraksh2

v1.0 - Algorithm Specification

Supriya Adhikary	Debayan Das	Abhinava De	Anindya Ganguly
Archisman Ghosh	Harry Hart	Angshuman Karmakar	Suparna Kundu
Chaoyun Li	Prajna Mahadev	Puja Mondal	Quinten Norga
Thales B. Paiva	Debranjana Pal	Subhradip Palchaudhuri	

Website: <https://sites.google.com/view/rudraksh2>

Date: 30th June 2026

Contents

1	Algorithm Specification and Description	4
1.1	Preliminaries and Notation	4
1.2	Specification of Rudraksh2.PKE	7
1.3	Specification of Rudraksh2.KEM	9
1.4	Rudraksh2 Parameter Sets	10
1.5	Design Rationale	10
1.6	Features	13
2	Performance Evaluation	16
2.1	Analysis, Considerations and Trade-offs	16
2.2	Transmission and Storage Cost	16
2.3	AVX2 Optimized Implementation	16
2.4	Performance of Rudraksh2	17
2.5	Performance of Rudraksh2 on Cortex-M4	19
2.5.1	Memory Optimization on Cortex-M4	19
2.6	Hardware Design	20
2.6.1	Proposed System Architecture	20
2.6.2	SM3 as a XOF	21
2.6.3	Resource Utilization	22
3	Security Analysis	23
3.1	Theoretical Security	23
3.1.1	IND-CCA security proof under the classical random oracle model	23
3.1.2	IND-CCA security proof under the quantum random oracle model	24
3.2	Practical Security	24
3.2.1	Attacks against LWE	24
3.2.2	Primal attack	24
3.2.3	Dual attack	25
3.2.4	Core-SVP hardness	25
3.2.5	Beyond core-SVP hardness	25
3.2.6	Attacks using decryption failures	26
3.3	Additional Security Properties	26
3.3.1	(Perfect) Forward Secrecy	26
3.3.2	Side-Channel Attack Resilience:	27
3.3.3	Multi-key Attack Resilience	27
3.3.4	(Temporary) Key Misuse Resilience	28
4	Advantages and Limitations	30

List of Algorithms

1	<code>PolyToBytes_q(a)</code>	4
2	<code>BytesToPoly_q(B)</code>	4
3	<code>SampleCBDpoly_η(seed_s, i)</code>	5
4	<code>SampleCBDvec_η(seed_s, i)</code>	5
5	<code>SampleMatrix(seed_A)</code>	6
6	<code>MinalB2.Encode()</code>	7
7	<code>MinalB2.Decode()</code>	7
8	<code>Rudraksh2.Encode()</code>	8
9	<code>Rudraksh2.Decode()</code>	8
10	<code>Rudraksh2.PKE.KeyGen()</code>	8
11	<code>Rudraksh2.PKE.Encrypt(pk, m; seed_r)</code>	9
12	<code>Rudraksh2.PKE.Decrypt(sk, c)</code>	9
13	<code>Rudraksh2.KEM.KeyGen()</code>	9
14	<code>Rudraksh2.KEM.Encaps(pk)</code>	10

15	<code>Rudraksh2.KEM.Decaps</code> ($\overline{\mathbf{sk}}$, c)	10
----	--	----

1 Algorithm Specification and Description

Rudraksh2 is an IND-CCA2-secure key-encapsulation mechanism (KEM), based on the module learning with errors (MLWE) problem [1]. Rudraksh2 is explicitly designed to be lightweight and particularly attractive for deployment in resource-constrained devices. The MLWE-based KEM is designed by constructing an IND-CPA-secure public-key encryption (PKE) scheme, and transforming it into an IND-CCA2-secure KEM through a slightly tweaked Fujisaki-Okamoto (FO) transform [18, 21]. A preliminary version of the design of this scheme was proposed in [25].

1.1 Preliminaries and Notation

Polynomial Rings For a positive integer q , the set of integers modulo q is denoted by $\mathbb{Z}_q$. We denote the quotient ring $\mathbb{Z}_q[x]/(x^n + 1)$ by $R_{q,n}$. We represent the ring with vectors of length ℓ by $R_{q,n}^\ell$. We denote the ring with matrices of size $m \times \ell$ by $R_{q,n}^{m \times \ell}$.

Polynomials, Vectors, and Matrices. We denote single polynomials by lower case letters (e.g., x), vectors by bold lower case letters (e.g., $\mathbf{y}$), and matrices by upper case letters (e.g., $\mathbf{Z}$). We denote the transpose of a vector $\mathbf{y}$ by $\mathbf{y}^\top$ and the transpose of a matrix $\mathbf{Z}$ by $\mathbf{Z}^\top$.

Bytes and Array of Bytes. We denote the set of 8-bit unsigned integers/bytes as $\mathbb{B}$. Consequently, a set of k -length arrays of bytes is represented with $\mathbb{B}^k$. A set of arbitrary-length arrays of bytes is denoted by $\mathbb{B}^*$. As the API accepts and returns byte arrays, it is necessary to transform polynomials and vectors between their polynomial representation and byte-array encoding. This is achieved using Algorithm 1 and Algorithm 2. To apply these algorithms to a vector of polynomials, simply apply them individually to each polynomial in the vector.

Algorithm 1 PolyToBytes_q(a)

Input: Polynomial a with coefficients $\{a_0, \dots, a_{n-1}\} \in \mathbb{Z}_q$

Output: Byte array $B \in \mathbb{B}^{\lceil \log_2 q \rceil n/8}$

```

1: for  $i = 0$  to  $n - 1$  do
2:   for  $j = 0$  to  $\lceil \log_2 q \rceil - 1$  do
3:      $b[\lceil \log_2 q \rceil i + j] \leftarrow (a_i/2^j) \bmod 2$ 
4: for  $i = 0$  to  $(\lceil \log_2 q \rceil n/8) - 1$  do
5:    $B[i] \leftarrow 0$ 
6:   for  $j = 0$  to  $7$  do
7:      $B[i] \leftarrow B[i] + b[8i + j] \cdot 2^j$ 
8: return  $B$ 
```

Algorithm 2 BytesToPoly_q(B)

Input: Byte array $B \in \mathbb{B}^{\lceil \log_2 q \rceil n/8}$

Output: Polynomial a with coefficients $\{a_0, \dots, a_{n-1}\} \in \mathbb{Z}_q$

```

1: for  $i = 0$  to  $(\lceil \log_2 q \rceil n/8) - 1$  do
2:   for  $j = 0$  to  $7$  do
3:      $b[8i + j] \leftarrow (B[i]/2^j) \bmod 2$ 
4: for  $i = 0$  to  $n - 1$  do
5:    $a_i \leftarrow 0$ 
6:   for  $j = 0$  to  $\lceil \log_2 q \rceil - 1$  do
7:      $a_i \leftarrow a_i + b[(\lceil \log_2 q \rceil i) + j] \cdot 2^j$ 
8: return  $a$ 
```

Symmetric-key primitives. Rudraksh2 relies on two cryptographic hash functions, $\mathcal{G}$ and $\mathcal{H}$, as well as a pseudorandom function PRF. Each of these primitives is instantiated using a distinct instance of an extendable-output function (XOF).

An XOF accepts an input of arbitrary length and produces an output of arbitrary length. Let the input be denoted by In with length In_len , and the output by Out with length Out_len . The function is defined as:

$$\text{Out} \leftarrow \text{XOF}(\text{In}, \text{In_len}, \text{Out_len}).$$

For the hash functions $\mathcal{G}$ and $\mathcal{H}$, the input lengths may vary, but their output lengths are fixed. Specifically, $\mathcal{G}$ outputs 2len_K bytes, while $\mathcal{H}$ outputs len_K bytes. The pseudorandom function PRF takes a fixed-length input and returns a fixed-length output that only depends on the parameters. It is defined as

$$\text{PRF} : \mathbb{B}^{\text{len}_K + 1} \longrightarrow \mathbb{B}^{\eta n/4}.$$

Distributions. If x is sampled from a set S according to distribution χ , then we represent it as $x \leftarrow \chi(S)$. We denote the uniform distribution on a set S by $\mathcal{U}(S)$. If x is generated from a seed seed_x and belongs to the set S and follows the distribution χ , then we represent it as $x \leftarrow \chi(S, \text{seed}_x)$.

Centered Binomial Distribution. We denote the centered binomial distribution (CBD) with parameter η , by β_η . In Rudraksh2, each n coefficient of error and secret vector of polynomials $\mathbf{s}(\in R_{q,n}^\ell)$, $\mathbf{s}'(\in R_{q,n}^\ell)$, $\mathbf{e}(\in R_{q,n}^\ell)$, $\mathbf{e}'(\in R_{q,n}^\ell)$, and $\mathbf{e}''(\in R_{q,n})$ are sampled from the centered binomial distribution B_η . B_η is defined as follows:

$$\text{randomly sample } (a_1, \dots, a_\eta, b_1, \dots, b_\eta) \leftarrow \mathcal{U}\{0, 1\}^{2\eta} \text{ and outputs } \sum_{i=1}^{\eta} (a_i - b_i) \bmod q.$$

A polynomial $s(\in R_{q,n})$ sampled from B_η using a seed seed_s follows Algorithm 3 and vector of polynomials $\mathbf{s}(\in R_{q,n}^\ell)$ sampled from B_η using a seed seed_s follows Algorithm 4.

Algorithm 3 $\text{SampleCBDpoly}_\eta(\text{seed}_s, i)$

Input: Seed seed_s and polynomial index i

Output: i th polynomial $s \in R_{q,n}$ with coefficients $\{s_0, \dots, s_{n-1}\} \in \mathbb{Z}_q$

- 1: $B \leftarrow \text{PRF}(\text{seed}_s || i) \in \mathbb{B}^{\eta n/4}$
 - 2: **for** $k = 0$ to $(\eta n/4) - 1$ **do**
 - 3: **for** $j = 0$ to 7 **do**
 - 4: $\beta[8k + j] \leftarrow (B[k]/2^j) \bmod 2$
 - 5: **for** $j = 0$ to $n - 1$ **do**
 - 6: $a \leftarrow \sum_{t=0}^{\eta-1} \beta[2j\eta + t]$
 - 7: $b \leftarrow \sum_{t=0}^{\eta-1} \beta[2j\eta + \eta + t]$
 - 8: $s_j \leftarrow (a - b) \bmod q$
 - 9: **return** s
-

Algorithm 4 $\text{SampleCBDvec}_\eta(\text{seed}_s, i)$

Input: Seed seed_s and polynomial start index i

Output: A vector of ℓ polynomials $\mathbf{s} \in R_{q,n}^\ell$ with polynomials $\{s[0], \dots, s[\ell - 1]\} \in R_{q,n}$

- 1: **for** $j = 0$ to $\ell - 1$ **do**
 - 2: $s[j] \leftarrow \text{SampleCBDpoly}_\eta(\text{seed}_s, i + j)$
 - 3: **return** $\mathbf{s}$
-

Modular reduction. Rudraksh2 uses a prime modulus q , and therefore, for constant-time prime reduction, it uses Montgomery reduction and Barrett reduction techniques for modular reduction [5].

Polynomial multiplication. Rudraksh2 is an MLWE-based KEM. Hence, NTT can be used for polynomial multiplication. NTT is used in LBC schemes due to its faster quasi-linear time complexity $O(n \log_2 n)$. For NTT, the modulus q needs to be a prime number with the primitive $2n$ -th root-of-unity in the prime field $\mathbb{Z}_q$.

NTT multiplication between two polynomials x and y from R_q is performed by $x \cdot y = \text{INTT}(\text{NTT}(x) \circ \text{NTT}(y))$, where “ $\circ$ ” is the point-wise multiplication.. Given ζ is the $2n$ -th primitive root of unity and $\omega = \zeta^2$, the $\text{NTT}(x) = \hat{x} = (\hat{x}_0, \hat{x}_1, \dots, \hat{x}_{n-1})$ and $\text{INTT}(\hat{x}) = x = (x_0, x_1, \dots, x_{n-1})$ are denoted by the following Eqs. (1) and (2):

$$\hat{x}_i = \sum_{j=1}^{n-1} x_j \zeta^{(2i+1)j} = \sum_{j=1}^{n-1} (x_j \zeta^j) \omega^{ij} \bmod q, \quad 0 \leq i \leq n-1. \quad (1)$$

$$x_j = 1/n \sum_{i=1}^{n-1} \hat{x}_i \zeta^{-(2i+1)j} = \zeta^j / n \sum_{i=1}^{n-1} \hat{x}_i \omega^{-ij} \bmod q, \quad 0 \leq j \leq n-1. \quad (2)$$

The Rudraksh2 scheme employs a public matrix $\mathbf{A}$ that is sampled uniformly at random from $R_{q,n}^{\ell \times \ell}$. Since $\mathbf{A}$ is uniformly random over $R_{q,n}^{\ell \times \ell}$, its transform $\text{NTT}(\mathbf{A}) = \hat{\mathbf{A}}$ is also uniformly random over the same domain. To facilitate efficient polynomial multiplication and improve performance, $\hat{\mathbf{A}}$ is sampled directly instead of sampling $\mathbf{A}$ and computing $\text{NTT}(\mathbf{A}) = \hat{\mathbf{A}}$. This sampling is performed deterministically from a uniform distribution using the seed $\text{seed}_{\mathbf{A}}$, as specified in Algorithm 5.

Algorithm 5 `SampleMatrix(seedA)`

Input: Byte array $\text{seed}_{\mathbf{A}} \in \mathbb{B}^{\text{len}_{\kappa}}$

Output: Matrix of polynomials $\mathbf{A} \in R_{q,n}^{\ell \times \ell}$

```

1: for  $i = 0$  to  $\ell - 1$  do
2:   for  $j = 0$  to  $\ell - 1$  do
3:      $k = 0$ 
4:      $\text{index} = 0$ 
5:     while  $\text{index} < (N - 1)$  do
6:        $B \leftarrow \text{PRF}(\text{seed}_{\mathbf{A}} || i || j || k) \in \mathbb{B}^{(\lceil \log_2 q \rceil n/8)}$ 
7:       for  $i_1 = 0$  to  $(\lceil \log_2 q \rceil n/8) - 1$  do
8:         for  $j_1 = 0$  to  $7$  do
9:            $b[8i_1 + j_1] \leftarrow (B[i_1] / 2^{j_1}) \bmod 2$ 
10:        for  $i_1 = 0$  to  $n - 1$  do
11:           $\text{temp} \leftarrow 0$ 
12:          for  $j_1 = 0$  to  $\lceil \log_2 q \rceil - 1$  do
13:             $\text{temp} \leftarrow \text{temp} + b[(\lceil \log_2 q \rceil i_1) + j_1] \cdot 2^{j_1}$ 
14:            if  $\text{temp} < q$  then
15:               $A[i][j]_{\text{index}} = \text{temp}$ 
16:               $\text{index} \leftarrow \text{index} + 1$ 
17:          if  $\text{index} < (N - 1)$  then
18:             $k \leftarrow k + 1$ 
19: return  $\mathbf{A}$ 

```

Compression and Decompression. Let us assume p is a power-of-two integer and $\log_2 p < \lceil \log_2 q \rceil$. Then Compress_p and Decompress_p are defined as follows.

- $\text{Compress}_p : R_q \rightarrow R_p$ function is defined as $\text{Compress}_p(x) = \lfloor \frac{p}{q} x \rfloor \bmod p$, $\lfloor \cdot \rfloor$ denotes the rounding operation.
- $\text{Decompress}_p : R_p \rightarrow R_q$ is defined as $\text{Decompress}_p(x') = \lfloor \frac{q}{p} x' \rfloor$.

If Compress_p or Decompress_p is applied to a polynomial or a vector of polynomials, then it is applied coefficientwise.

The main reason for defining the Compress_q and Decompress_q functions is to be able to discard some low-order bits in the ciphertext, which do not have much effect on the correctness probability of decryption – thus reducing the size of ciphertexts.

Encoding and decoding using extended Minal codes. To improve the correctness probability, we define an extension of 2D Minal codes [30] for error correction that encodes 4 bits into two coefficients in $\mathbb{Z}_q$. Minal codes [30] are d -dimensional error correction codes with coefficients in $\mathbb{Z}_q$. Although inspired by lattice codes, they differ in one critical aspect: Minal codes are periodic in $\mathbb{Z}_q^d$, which makes them useful for lattice-based schemes where operations are typically done modulo a prime q . In our case, we are particularly interested in 2D Minal codes, which admit efficient decoders. For a prime q , and a non-negative integer $\beta < q/2$, the 2D Minal code with parameter β is the set $\mathcal{C} = \{\mathbf{G}\mathbf{m} + q\mathbf{z} : \mathbf{z} \in \mathbb{Z}^d \text{ and } \mathbf{m} \in \mathbb{Z}_2^2\}$, where $\mathbf{G} \in \mathbb{Z}^{2 \times 2}$ is the circulant matrix generated by $[\lfloor q/2 \rfloor, \beta]$. Matrix $\mathbf{G}$ is called the generator matrix of $\mathcal{C}$ and β is called the tailoring parameter. We say that $\mathbf{c}$ encodes a 2-bit message $\mathbf{m} \in \mathbb{Z}_2^2$ when $\mathbf{c} = \mathbf{G}\mathbf{m} + q\mathbf{z}$, for some $\mathbf{z} \in \mathbb{Z}^d$.

Since 2D Minal codes encoded 2 bits into 2 coefficients, they can only be used directly in KEMs for which the message size μ is exactly equal to the degree n , used to define the ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$. However, in Rudraksh2, we use $\mu = 2n$. To enable the smaller polynomial degrees, we define an extension of Minal codes called B2 Minal, enabling us to encode 4 bits into 2 coefficients in $\mathbb{Z}_q$ as follows. First, let $\mathbf{G}_1$ be the generator matrix of the 2D pure Minal code in $\mathbb{Z}_{\lfloor q/2 \rfloor}$. We then build a generator matrix $\mathbf{G}_2$ of the B2-2D-Minal code as

$$\mathbf{G}_2 = \left[\begin{array}{cc|c} \lfloor q/2 \rfloor & 0 & \mathbf{G}_1 \\ 0 & \lfloor q/2 \rfloor & \end{array} \right] = \left[\begin{array}{cc|cc} \lfloor q/2 \rfloor & 0 & \lfloor q/4 \rfloor & \beta \\ 0 & \lfloor q/2 \rfloor & \beta & \lfloor q/4 \rfloor \end{array} \right].$$

Algorithm 6 shows how the encoding operation of a 4-bit message $\mathbf{m}$ corresponds to the computation of $\mathbf{c} = \mathbf{G}_2\mathbf{m} \in \mathbb{Z}_q^2$. The decoding of a noisy codeword $\mathbf{c}$, show as Algorithm 7, is then done in two phases. First we use a regular 2D Minal decoder to obtain the last two bits (m_2, m_3) of the message, as these were encoded based on the 2D Minal code generated by $\mathbf{G}_1$. Then, in the second phase, we obtain the first two bits (m_0, m_1) of the message by checking which of the 4 possibilities for (m_0, m_1) yields the codeword $\mathbf{G}_2[m_0, m_1, m_2, m_3]^\top$ closest to the target noisy codeword $\mathbf{c}$.

Algorithm 6 `MinalB2.Encode()`

Input: Message bits $\mathbf{m} \in \mathbb{Z}_2^4$

Output: Codeword $\mathbf{c} \in \mathbb{Z}_q^2$

- 1: $\mathbf{c} \leftarrow \mathbf{G}_2\mathbf{m} \in \mathbb{Z}_q^2$
 - 2: **return** $\mathbf{c}$
-

Algorithm 7 `MinalB2.Decode()`

Input: Noisy codeword $\mathbf{c} \in \mathbb{Z}_q^2$

Output: Message bits $\mathbf{m} \in \mathbb{Z}_2^4$

- 1: $\triangleright$ **Phase 1:** Decode bits (m_2, m_3)
 - 2: $\mathbf{c}' \leftarrow \mathbf{c} \bmod \lfloor q/2 \rfloor$ $\triangleright \mathbf{c}'$ is a regular Minal codeword in $\mathbb{Z}_{\lfloor q/2 \rfloor}^2$
 - 3: $\text{min_dist} \leftarrow \infty$
 - 4: **for** $\mathbf{v}$ in $\{(0, 0), (0, 1), (1, 0), (1, 1)\}$ **do** $\triangleright$ Minimum distance decoding for pure Minal
 - 5: $d \leftarrow \|\mathbf{c}' - \mathbf{G}_1\mathbf{v} \bmod \pm \lfloor q/2 \rfloor\|_2$
 - 6: **if** $d < \text{min_dist}$ **then**
 - 7: $\text{min_dist} \leftarrow d$
 - 8: $(m_2, m_3) \leftarrow \mathbf{v}$
 - 9: $\triangleright$ **Phase 2:** Decode bits (m_0, m_1)
 - 10: $\mathbf{c}_{\text{best}} \leftarrow \mathbf{G}_1 [m_2 \ m_3]^\top$
 - 11: $(\delta_x, \delta_y) \leftarrow \mathbf{c} - \mathbf{c}_{\text{best}} \bmod \pm q$
 - 12: $m_0 \leftarrow \lfloor |\delta_x| / \lfloor q/4 \rfloor \rfloor$
 - 13: $m_1 \leftarrow \lfloor |\delta_y| / \lfloor q/4 \rfloor \rfloor$
 - 14: **return** $\mathbf{m} = (m_0, m_1, m_2, m_3)$
-

1.2 Specification of Rudraksh2.PKE

Rudraksh2 is based on the MLWE hard problem [1], a variant of the LWE problem [31], which has previously been used in the construction of Kyber [12]. The Rudraksh2 public-key encryption

Algorithm 8 `Rudraksh2.Encode()`

Input: Message bits $\mathbf{m} \in \mathbb{Z}_2^\mu$ (where $\mu = 2n$)**Output:** Codeword $\mathbf{c} \in \mathbb{Z}_q^n$

- 1: $\mathbf{c} \leftarrow \mathbf{0} \in \mathbb{Z}_q^n$
 - 2: **for** $i = 0$ to $\mu/4$ **do**
 - 3: $\mathbf{c}[i], \mathbf{c}[i + n/2] \leftarrow \text{MinalB2.Encode}(\mathbf{m}[4i], \mathbf{m}[4i + 1], \mathbf{m}[4i + 2], \mathbf{m}[4i + 3]) \in \mathbb{Z}_q^2$
 - 4: **return** $\mathbf{c}$
-

Algorithm 9 `Rudraksh2.Decode()`

Input: Noisy codeword $\mathbf{c} \in \mathbb{Z}_q^n$ **Output:** Message bits $\mathbf{m} \in \mathbb{Z}_2^\mu$ (where $\mu = 2n$)

- 1: $\mathbf{c} \leftarrow \mathbf{0} \in \mathbb{Z}_q^n$
 - 2: **for** $i = 0$ to $\mu/4$ **do**
 - 3: $\mathbf{m}[4i], \mathbf{m}[4i + 1], \mathbf{m}[4i + 2], \mathbf{m}[4i + 3] \leftarrow \text{MinalB2.Decode}(\mathbf{c}[i], \mathbf{c}[i + n/2]) \in \mathbb{Z}_q^4$
 - 4: **return** $\mathbf{c}$
-

(PKE) scheme is indistinguishable under chosen plaintext attacks (IND-CPA) and consists of three algorithms: `Rudraksh2.PKE.KeyGen` (Algorithm 10), `Rudraksh2.PKE.Encrypt` (Algorithm 11) and `Rudraksh2.PKE.Decrypt` (Algorithm 12).

Algorithm 10 `Rudraksh2.PKE.KeyGen()`

*Generates a public encryption key and corresponding secret decryption key.***Output:** pk, sk

- 1: $(\text{seed}_{\mathbf{A}}, \text{seed}_{\text{se}}) \leftarrow \mathcal{U}(\{0, 1\}^{\text{len}_K} \times \{0, 1\}^{\text{len}_K})$
 - 2: $\hat{\mathbf{A}} \leftarrow \text{SampleMatrix}(\text{seed}_{\mathbf{A}}) \in R_{q,n}^{(\ell \times \ell)}$ $\triangleright \hat{\mathbf{A}} = \text{NTT}(\mathbf{A})$
 - 3: $\mathbf{s} \leftarrow \text{SampleCBDvec}_\eta(\text{seed}_{\text{se}}, 0) \in R_{q,n}^\ell$
 - 4: $\mathbf{e} \leftarrow \text{SampleCBDvec}_\eta(\text{seed}_{\text{se}}, \ell) \in R_{q,n}^\ell$
 - 5: $\hat{\mathbf{s}} \leftarrow \text{NTT}(\mathbf{s}) \in R_{q,n}^\ell$
 - 6: $\hat{\mathbf{e}} \leftarrow \text{NTT}(\mathbf{e}) \in R_{q,n}^\ell$
 - 7: $\hat{\mathbf{b}} \leftarrow (\hat{\mathbf{A}} \circ \hat{\mathbf{s}} + \hat{\mathbf{e}}) \in R_{q,n}^\ell$
 - 8: $\text{pk} \leftarrow (\text{seed}_{\mathbf{A}} \parallel \text{PolyToBytes}_q(\hat{\mathbf{b}}))$
 - 9: $\text{sk} \leftarrow \text{PolyToBytes}_q(\hat{\mathbf{s}})$
 - 10: **return** (pk, sk)
-

Algorithm 11 `Rudraksh2.PKE.Encrypt`(pk, m; seed_r)

Produces a ciphertext by encrypting a plaintext message, encryption key and random seed.

Input: pk, m; seed_r

Output: c

```
1: (seedA ||  $\hat{\mathbf{b}}$ )  $\leftarrow$  BytesToPolyq(pk)
2:  $\hat{\mathbf{A}} \leftarrow$  SampleMatrix(seedA)  $\in R_{q,n}^{(\ell \times \ell)}$ 
3: if seedr is not specified then
4:   seedr  $\leftarrow \mathcal{U}(\{0, 1\}^{256})$ 
5:  $\mathbf{s}' \leftarrow$  SampleCBDvecη(seedr, 0)  $\in R_{q,n}^\ell$ 
6:  $\mathbf{e}' \leftarrow$  SampleCBDvecη(seedr, ℓ)  $\in R_{q,n}^\ell$ 
7:  $\mathbf{e}'' \leftarrow$  SampleCBDpolyη(seedr, 2ℓ)  $\in R_{q,n}$ 
8:  $\hat{\mathbf{s}}' \leftarrow \text{NTT}(\mathbf{s}') \in R_{q,n}^\ell$ 
9:  $\hat{\mathbf{b}}' \leftarrow \hat{\mathbf{A}}^T \circ \hat{\mathbf{s}}' \in R_{q,n}^\ell$ 
10:  $\mathbf{b}' \leftarrow (\text{INTT}(\hat{\mathbf{b}}') + \mathbf{e}') \in R_{q,n}$ 
11:  $\hat{c}_m \leftarrow \hat{\mathbf{b}}'^T \circ \hat{\mathbf{s}}' \in R_{q,n}$ 
12:  $c_m \leftarrow \text{INTT}(\hat{c}_m) + \mathbf{e}'' + \text{Rudraksh2.Encode}(m) \in R_{q,n}$ 
13:  $\mathbf{u} \leftarrow \text{Compress}_p(\mathbf{b}') \in R_{p,n}^\ell$ 
14:  $v \leftarrow \text{Compress}_t(c_m) \in R_{t,n}$ 
15:  $c \leftarrow \text{PolyToBytes}_p(\mathbf{u}) || \text{PolyToBytes}_t(v)$ 
16: return c
```

Algorithm 12 `Rudraksh2.PKE.Decrypt`(sk, c)

Decrypts a ciphertext using the decryption key.

Input: sk, c

Output: m'

```
1:  $\hat{\mathbf{s}} \leftarrow \text{BytesToPoly}_q(\text{sk})$ 
2:  $(\mathbf{u}, v) \leftarrow \text{BytesToPoly}_{p,t}(c)$ 
3:  $\mathbf{u}' \leftarrow \text{Decompress}_p(\mathbf{u}) \in R_{q,n}^\ell$ 
4:  $v' \leftarrow \text{Decompress}_t(v) \in R_{q,n}$ 
5:  $\hat{\mathbf{u}}' \leftarrow \text{NTT}(\mathbf{u}') \in R_{q,n}^\ell$ 
6:  $\hat{u}'' \leftarrow \hat{\mathbf{u}}'^T \circ \hat{\mathbf{s}} \in R_{q,n}$ 
7:  $m'' \leftarrow v' - \text{INTT}(\hat{u}'') \in R_{q,n}$ 
8:  $m' \leftarrow \text{Rudraksh2.Decode}(m'') \in R_{2^B}$ 
9: return m'
```

1.3 Specification of Rudraksh2.KEM

We convert the IND-CPA Rudraksh.PKE to an indistinguishable under adaptive chosen ciphertext attack (IND-CCA2) KEM utilizing a variant of Fujisaki–Okamoto (FO) transformation proposed in [21] and also used in FrodoKEM [11]. The Rudraksh2 KEM scheme consists of three algorithms: `Rudraksh2.KEM.KeyGen` (Algorithm 13), `Rudraksh2.PKE.Encapsulation` (Algorithm 14) and `Rudraksh2.PKE.Decaps` (Algorithm 15).

Algorithm 13 `Rudraksh2.KEM.KeyGen`()

Output: $\overline{\text{pk}}, \overline{\text{sk}}$

```
1: (pk, sk) := ((seedA, b), s) := Rudraksh2.PKE.KeyGen()
2: pkh :=  $\mathcal{H}(\text{pk})$ 
3:  $z \leftarrow \mathcal{U}(\{0, 1\}^{\text{len}_K})$ 
4:  $\overline{\text{pk}} \leftarrow \text{pk}$ 
5:  $\overline{\text{sk}} \leftarrow (\text{sk} || \text{pk} || \text{pkh} || z)$ 
6: return ( $\overline{\text{pk}}, \overline{\text{sk}}$ )
```

Algorithm 14 `Rudraksh2.KEM.Encaps`($\overline{\text{pk}}$)

Input: $\overline{\text{pk}}$ **Output:** c, K

- 1: $m \leftarrow \mathcal{U}(\{0, 1\}^{\text{len}_K})$
 - 2: $(K, \text{seed}_r) \leftarrow \mathcal{G}(\mathcal{H}(\text{pk}), m)$
 - 3: $c \leftarrow \text{Rudraksh2.PKE.Encryption}(\text{pk}, m; \text{seed}_r)$
 - 4: **return** (c, K)
-

Algorithm 15 `Rudraksh2.KEM.Decaps`($\overline{\text{sk}}, c$)

Input: $\overline{\text{sk}}, c$ **Output:** K

- 1: $(\text{sk}||\text{pk}||\text{pkh}||z) \leftarrow \overline{\text{sk}}$
 - 2: $m' \leftarrow \text{Rudraksh2.PKE.Decrypt}(\text{sk}, c)$
 - 3: $(K', \text{seed}_{r'}) \leftarrow \mathcal{G}(\text{pkh}, m') \in \{0, 1\}^{\text{len}_K} \times \{0, 1\}^{\text{len}_K}$
 - 4: $c_* \leftarrow \text{Rudraksh2.PKE.Encryption}(\text{pk}, m'; \text{seed}_{r'})$
 - 5: $K'' \leftarrow \mathcal{H}(c, z)$
 - 6: **if** $c = c_*$ **then**
 - 7: **return** $K \leftarrow K'$
 - 8: **else**
 - 9: **return** $K \leftarrow K''$
 - 10: **return** K
-

1.4 Rudraksh2 Parameter Sets

Parameter sets of Rudraksh2, corresponding to classical security strengths of (at least) 128, 256, and 512 bits and quantum security strength of (at least) 80, 128 and 256 bits are presented in Table 1. We propose two parameter set variants: Rudraksh2-I and Rudraksh2-II:

Table 1: Parameter sets of Rudraksh2

	Module Parameter		Primary modulus		Compression modulus		CBD parameter	Encoding	Seed size	Failure probability
	l	n	q	$\lceil \log_2 q \rceil$	p	t	η	(B, β)	len_K	$(\log_2)$
Rudraksh-128-I	9	64	3329	12	2^{12}	2^6	2	(2, 220)	128	-100
Rudraksh-128-II	9	64	4001	12	2^9	2^7	1	(2, 270)	128	-100
Rudraksh-256-I	9	128	3329	12	2^{11}	2^9	1	(2, 220)	256	-161
Rudraksh-256-II	9	128	4001	12	2^{11}	2^5	1	(2, 250)	256	-167
Rudraksh-512-I	8	256	7681	13	2^{13}	2^7	2	(2, 510)	512	-160
Rudraksh-512-II	9	256	4001	12	2^{12}	2^8	1	(2, 270)	512	-160

1.5 Design Rationale

The design of Rudraksh2 is inspired by the long R&D efforts in the field of learning with errors, especially the MLWE-based schemes. One of our primary objectives for this design is to make Rudraksh2 suitable for a wide range of use cases, by bridging the gap between theoretical security and practical performance constraints. In particular, Rudraksh2 can be suitably used for resource-constrained devices such as Internet of Things (IoT) as well as high-performance devices such as servers.

Module LWE Rudraksh2 is based on the module learning with errors (MLWE) quantum-hard problem, a variant of the LWE problem. The central operation of almost any LWE-based cryptographic scheme is to calculate $A \cdot s + e \in \mathbb{Z}_q^m$, where $A \in \mathbb{Z}_q^{m \times n}$ is a (large) public matrix and $s \in \mathbb{Z}_q^n$ and $e \in \mathbb{Z}_q^n$ are secret vectors of noise and errors, respectively. In Regev’s proposal of LWE [31], every entry in the matrix $A \in \mathbb{Z}_q^{m \times n}$ is sampled independently and uniformly from the integers modulo q . It offers the most *conservative* security reduction with very few structural assumptions, and is used

by FrodoKEM [11]. The major disadvantage of this design is that, due to the expensive matrix-vector multiplications, schemes based on this assumption are often very slow.

In a variant of LWE over a ring, RLWE [27], the matrix $\mathbf{A}$ is replaced by a single polynomial a in a ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$, reducing the size by a factor of n compared to standard LWE. It also allows the use of the Number Theoretic Transform (NTT), which reduces the complexity of polynomial multiplication to $O(n \log n)$. Post-quantum KEMs NewHope [5] and LAC [26] and digital signature scheme BLISS [16] are prominent examples of this approach.

Module LWE (MLWE) provides a trade-off between the previous two approaches by using a matrix $\mathbf{A}$ of size $l \times l$, where each entry is a polynomial of degree n in the ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$, capturing the best properties of both LWE and RLWE. One of the primary advantages of MLWE is its flexibility: it allows granular, incremental scaling of security. Since the security is proportional to the rank of the underlying matrix *i.e.* $l \times n$, one can increase one (or both) parameters to reach higher security categories. Currently, there does not exist any attack that can exploit the structure of A to its advantage. This framework has been used extensively in modern LWE-based designs, such as Kyber (ML-KEM) [12, 7] and Dilithium (ML-DSA) [17].

Parameter Space Exploration In the MLWE framework, the same security level can be achieved through different combinations of polynomial degree n and matrix parameter l . However, both parameters influence the performance characteristics of a scheme, including area and energy requirements, and degree of parallelism. For example, in any lattice-based KEM, the polynomial multiplication (NTT) is the primary computational bottleneck. The resource consumption in terms of Look-Up Tables (LUTs) and Flip-Flops (FFs) is directly proportional to the degree n . The size of the polynomial n also controls the interleaving of communication and computation, and memory requirements. Depending on the size of n , encapsulation and decapsulation modules can begin computation as soon as the first polynomial of a public key or ciphertext is received from the network. For example, for a smaller n , this allows the scheme to operate effectively even in networks with very limited instantaneous bandwidth, such as Bluetooth Low Energy (BLE) or ZigBee.

While most schemes like Kyber and Dilithium utilize a fixed degree of $n = 256$, our module-space exploration strategy uncovered a wide array of polynomial sizes ($n = 64, 128, 256$) and their corresponding module ranks (l) to find the *sweet spot* balancing security and practicality. For our design, we propose three different values of n . For the lowest security tier, $n = 64$ helps to minimize the storage and area requirements. For higher security categories, we propose $n = 128$ (Medium) and $n = 256$ (High), that provides a granular mechanism to maintain a compact hardware footprint without allowing the module rank l to grow very large.

Multiple Embedding of Secret bits A core design choice to improve the efficiency of Rudraksh2 is the use of high-density secret embedding. While traditional lattice-based schemes like Kyber and Saber typically encode only 1 bit of the shared secret per polynomial coefficient ($B = 1$), Rudraksh2 is designed to embed 2 bits per coefficient ($B = 2$). This choice ensures that the constituent polynomial sizes are perfectly sized for the required secret. For the 128-bit security tier, a polynomial of degree $n = 64$ with $B = 2$ produces exactly the 128 bits of entropy required. In $B = 1$ designs with $n = 256$, only half the coefficients are strictly necessary for a 128-bit secret, leading to repetition of the secrets, which, although slightly reduces the failure probability, larger polynomials affect hardware resources, and efficiency on redundant arithmetic. Our choice of embedding multiple secret bits per polynomial coefficient allows us to significantly reduce the polynomial degree n , which directly shrinks the area of the Number Theoretic Transform (NTT) unit and reduces coefficient storage requirements in memory-constrained IoT devices.

Tiered Approach to Tradeoff Security and Resource Consumption. Rudraksh2 adopts a new tiered approach to scale security. Unlike established standards that maintain a fixed polynomial degree (typically $n = 256$) and scale by increasing the module rank l , Rudraksh2 scales by varying the ring size n while keeping l optimized and fixed. While increasing the module rank l is the conventional scaling method, it does not improve the efficiency of the base arithmetic unit. Rudraksh’s n -varying strategy ensures that even at the highest security levels, the system remains highly parallelizable. For higher security, where low latency is more important than power consumption, one can instantiate l^2 multipliers in parallel. This translates to the fact that the efficiency of the whole scheme is dependent

on the efficiency of the base multiplier. Therefore, in this case, the target is to make the base multiplier as efficient as possible. For the lowest security level **Rudraksh-128** as shown in Table. 1, we utilize $n = 64$ to achieve the minimal resources, such as hardware area, memory footprint, etc. Since this security level is mostly targeted at low-power and resource-constrained devices like IoT, medical sensors, etc., reducing resource usage is the priority. For medium and high security *i.e* **Rudraksh-256** and **Rudraksh-512**, the n scales to $n = 128$ and $n = 256$ respectively. This ensures that computational cycles and power consumption remain proportional to the target security level. Since, in these scenarios, low latency and efficiency get priority over resource consumption, for the higher-parameter versions ($n = 256$), we optimize other parameters like the Centered Binomial Distribution (CBD) to facilitate faster parallel multiplications.

NTT for Efficient Polynomial Multiplication. Polynomial multiplication is a core computational bottleneck in any lattice-based cryptographic scheme. Rudraksh2 uses Number Theoretic Transform (NTT) for polynomial multiplications. Although there are other options such as Toom-Cook and Karatsuba polynomial multiplication, our parameter choices is suitable for NTT-based multiplication. Apart from this, NTT does not require additional storage, and one can directly sample in the NTT domain. These properties are very helpful for reducing resource usage and improving efficiency. Apart from this, for NTT multiplications, we don't have to complete the sequence of forward NTT-pointwise multiplication-inverse transform. We can often defer the reverse transformation for later. For example, during the KeyGen algorithm, while generating the $A \cdot s + e$, we can perform only the forward NTT operation of the error vector e and the secret vector s . The matrix vector multiplication between A and s is just pointwise multiplication between the constituent (NTT transformed) polynomials of A and s . Instead of applying inverse NTT on this result, the result is directly added to the $NTT(e)$. The final addition is also kept in the NTT domain, deferring the last inverse NTT operation. Similarly, the summation of polynomial products in matrix-vector operations can be performed entirely within the NTT domain. Instead of an inverse transform after every product, Rudraksh2 defers the INTT until the final summation is complete. This reduces the number of INTT operations from l^2 down to as few as 1, drastically lowering latency and energy consumption. For these reasons, we chose NTT over other polynomial multiplication algorithms.

CCA2 Security. While many lightweight applications can operate with passive (IND-CPA) security, our goal here is to design a Key-Encapsulation Mechanism (KEM) to be actively secure against Chosen-Ciphertext Attacks (IND-CCA2). This level of security is essential to allow for static-key usage, where a single public key can be safely reused for multiple sessions without the risk of an adversary extracting the secret key through multiple ciphertext (possibly malformed) queries. Rudraksh2 achieves IND-CCA2 security by applying a refined variant of the Fujisaki-Okamoto (FO) transform to its underlying CPA-secure Public-Key Encryption (PKE) scheme. Rudraksh2 utilizes a variant of the FO transform that performs re-encryption during decapsulation. After decrypting the ciphertext to obtain a message m' , the decapsulation algorithm re-encrypts m' to generate a "candidate" ciphertext c_* . The session key is only released if this candidate ciphertext exactly matches the received ciphertext ($c = c_*$). This ensures that an adversary cannot gain information about the secret key by observing the system's reaction to invalid or manipulated ciphertexts. A critical aspect of CCA security in lattice-based KEMs is the management of decryption failures. Here, the FO transform is designed to be robust against failure-based attacks by ensuring that any mismatch—whether caused by an attack or a legitimate (though rare) decryption failure results in a pseudo-random key K'' rather than a specific error. This ensures the adversary cannot extract any information about the secret key through inducing failures.

Achieving negligible failure probability using extended 2D Minal codes. As for most lattice-based KEMs, the decryption algorithm of Rudraksh2 needs to correct a noisy version of the encapsulated message. Rudraksh2 aims at two targets for decryption failure probability: 2^{-100} for security level 1 and 2^{-160} for the higher security levels 2 and 3. To achieve these values, we first conducted a parameter search for combinations that ensured mathematical security, and then, for selected parameters, we tailored the B2 Minal codes parameter β to minimize decryption failure probability. The analysis of the decryption failure probability is described next.

The decryption of a ciphertext $(\mathbf{u}, v)$ using secret key $\mathbf{s}$ starts by decompressing the ciphertext components to obtain $\mathbf{u}' = \text{Decompress}(\mathbf{u}, d_{\mathbf{u}})$ and $v' = \text{Decompress}(v, d_v)$, which is followed by the

computation of $m' = \text{Decode}(v' - \langle \mathbf{u}', \mathbf{s} \rangle)$. Let $\Delta \mathbf{u}$ and Δv be the errors introduced by compressing and decompressing the first and second parts of the ciphertext, respectively. Then we can write $m'' = v' - \langle \mathbf{u}', \mathbf{s} \rangle = \text{Encode}(\text{msg}) + \Delta m$, where the accumulated noise polynomial Δm is given by $\Delta m = \langle \mathbf{e}, \mathbf{r} \rangle - \langle \mathbf{s}, \mathbf{e}_1 + \Delta \mathbf{u} \rangle + e_2 + \Delta v$. Therefore, using the B2-Minal codes for error correction, we can apply the union bound to obtain an upper bound on the decryption failure probability (DFP) as

$$\begin{aligned} \text{DFP} &\leq \sum_{i=0}^{n/2-1} \Pr(\text{MinalB2.Decode}(m''[i, i+n/2]) = \text{msg}[4i, 4i+1, 4i+2, 4i+3]) \\ &= \frac{n}{2} \Pr(\text{MinalB2.Decode}(m''[0, n/2]) = \text{msg}[0, 1, 2, 3]) \\ &= \frac{n}{2} \Pr(\text{MinalB2.Decode}(\text{MinalB2.Encode}(\text{msg}[0, 1, 2, 3]) + \Delta m[0, n/2]) = \text{msg}[0, 1, 2, 3]). \end{aligned}$$

The computation of the failure probability can then be done as follows. First we compute the joint distribution $\Pr(\Delta m[0, n/2])$ of the noise on coordinates 0 and $n/2$. Then, for each possible 4-tuple of message bits $\text{msg}[0, 1, 2, 3]$, we iterate over all possible values of the noise $\Delta m[0, n/2]$ and check if decoding fails or not, updating the DFP accordingly. The most expensive step is the computation of the joint distribution $\Pr(\Delta m[0, n/2])$, which can be done following the analysis of Minal codes applied to ML-KEM [30]. In particular, as done in the analysis of regular Minal codes, we need to use FFT-based convolutions with arbitrary precision to efficiently compute the full computation of $\Delta m[0, n/2]$. Therefore, to facilitate the security analysis of Rudraksh2, all scripts used to compute the decryption failure probability will be made publicly available.

1.6 Features

The Rudraksh2 design exhibits the following features that set it apart from existing designs. Its design features ensure that it has less complicated security assumptions, minimizes the risk of physical attacks, is easy to implement on a wide range of platforms, and is easy to deploy in the real world.

Innovativeness. Rudraksh2 introduces a novel entropy-matched tiered design that departs from the "fixed-ring" convention of established standards. By varying the polynomial degree ($n = 64, 128, 256$) to match the target secret's entropy, it eliminates the redundancy found in traditional designs and meets the low power requirements of the target applications. Rudraksh2 has been designed to be suitable for low-power and resource-constrained devices in mind. At this moment, none of the PQC schemes have been designed for low-power and resource-constrained devices. The design of Rudraksh2 fills this gap.

Simplicity. The construction is deliberately modular and hardware-friendly. Rudraksh2 is based on the well-known Module-LWE, without any aggressive security assumptions, and is free of complicated security reductions. Rudraksh2 also uses simple prime moduli, which enable simpler shift and add modular reduction (k-reduction) compared to Montgomery and Barrett reduction. Apart from this, the other components of Rudraksh2 are also simple and easy to implement securely. The simplicity of this design is further enhanced by keeping the primary modulus $q = 4001$ and the module rank l constant (or nearly constant) across all security tiers. Because q is universal, the complex modular reduction logic, the most resource-intensive part of the arithmetic unit, remains identical for every security level. Even for other values of q , it requires only changing the roots-of-unity (and their universe), which can be done on the fly, for example, using a multiplexer in hardware. Furthermore, keeping l constant ensures that the high-level structure remains the same. Apart from this, the other components of Rudraksh2 are also simple and easy to implement securely, or can be duplicated for different security strengths.

Flexibility. Unlike many cryptographic schemes that rely on specialized hardware instructions such as AES-NI for symmetric-key cryptography acceleration or complex floating-point arithmetic units, Rudraksh2 is designed to be implemented using pure integer-based modular arithmetic and without the need for complex instruction-set arithmetic. This scheme is highly portable from high-end processors to low-end microcontrollers. The core of the Rudraksh2 flexibility lies in its multiplier-centric design. A single NTT-based multiplier unit designed for the highest security parameter ($n = 256$) is natively backward-compatible with the lower tiers ($n = 64$ and $n = 128$). This allows a single software or hardware implementation to support multiple security levels by simply adjusting the transform length,

providing a significant advantage for manufacturers that need to meet various post-quantum security requirements on a single chip.

Compatibility. Based on the well-known Module-LWE (MLWE) hard problem, Rudraksh2 maintains mathematical alignment with other PQC standards. Because it shares the same core MLWE structure as NIST-standardized schemes, Rudraksh2 can leverage existing hardware IP and specialized accelerators. Hardware modules developed for Kyber or Dilithium, such as NTT engines, modular reduction units, and matrix-vector multiplication state machines, can be reused or adapted for Rudraksh2 with minimal overhead. This significantly lowers the barrier to adoption for organizations that have already invested in lattice-based hardware infrastructure. As described above, Rudraksh2 also supports native cross-standard compatibility. A hardware accelerator provisioned for a 256-degree polynomial (the standard for Kyber) can natively support the various tiers of Rudraksh. Also, by maintaining structural consistency with international lattice standards (such as the use of prime moduli and NTT-friendly rings), Rudraksh2 is highly compatible with the data structures of emerging post-quantum protocols like TLS 1.3, IKEv2, and SSH. Its compact keys and ciphertexts are designed to fit within the existing packet frames of low-power wireless protocols, enabling a smooth migration path for IoT deployments where hardware remains static, but security must be upgraded.

Extensibility. The design and algorithms developed for Rudraksh2 can be extended beyond Key Encapsulation. The core arithmetic IP, specifically the reconfigurable NTT multiplier and the k -reduction unit, serves as a high-performance foundation for other lattice-based primitives. This hardware can be readily repurposed to implement Digital Signatures (e.g., Dilithium-like constructions with smaller polynomials) and Authenticated Key Exchange (AKE) with minimal additional logic. The tiered scaling approach ($n = 64, 128, 256$) allows the scheme to be extended into complex multi-party or group key agreement scenarios. The high efficiency of the $n = 64$ and $n = 128$ rings provides an ideal execution environment for complex future primitives, including Identity-Based Encryption (IBE) and Lightweight Functional Encryption, which rely on multiple small polynomial operations.

Performance Advantages on Variant Platforms. The unique "Small-Ring, High-Rank" architectural philosophy of Rudraksh2 ensures superior performance across a diverse range of computational environments,

- **Hardware (FPGA/ASIC):** Initial design of Rudraksh achieves a low hardware footprint, requiring as few as 2,813 LUTs and 1.5 BRAMs on a Xilinx Artix-7 FPGA. This represents a $\sim 3\times$ reduction in area compared to the most area-optimized implementations of Kyber-512 [25]. Furthermore, the simplified k -reduction logic allows the hardware to operate at a 63%–76% higher frequency than standard lattice-based KEMs, providing a superior throughput-to-area ratio for embedded security modules.
- **Massively Parallel Platforms (GPUs):** The modular structure of Rudraksh2 is inherently optimized for the SIMT (Single Instruction, Multiple Threads) architecture of modern GPUs. Because the Module-LWE operation involves independent matrix-vector multiplications, the workload is highly parallelizable. Each small $n = 64$ or $n = 128$ polynomial multiplier can be mapped directly to a single GPU Block or thread group. By utilizing the thousands of cores available in a GPU, a gateway server can process thousands of concurrent Rudraksh2 handshakes simultaneously. The small ring size ensures that twiddle factors and coefficients fit comfortably within high-speed Shared Memory (L1 cache), minimizing costly global memory accesses.
- **Microcontrollers (Cortex-M0/M4):** On small microcontrollers, the smaller polynomial rings ($n = 64, 128$) offer significant memory stack savings. Unlike standard $n = 256$ schemes that require large temporary buffers for NTT twiddle factors, Rudraksh2 operates with a minimal memory footprint, making it suitable for devices with extremely limited SRAM. Additionally, due to the Just-In-Time (JIT) [23] matrix generation strategy. Rather than expanding and storing the entire public matrix $\mathbf{A}$ in memory, Rudraksh2 generates each polynomial element of the matrix on-the-fly during the matrix-vector multiplication process. This reduces the instantaneous memory requirement for the full matrix storage to the storage of just a single polynomial, allowing the scheme to operate comfortably within the extremely limited SRAM of M0+ and M4 class devices.

- **General-Purpose CPUs (x86/ARM):** Rudraksh2 is designed for maximum portability and does not rely on platform-specific hardware like AES-NI or floating-point units. However, it can exploit modern vector instructions (SIMD) such as SSE, AVX2, and AVX-512 on x86, or NEON on ARM. For example, the smaller polynomial rings ($n = 64, 128$) allow multiple coefficients, or even entire polynomials, to be packed into a single vector register. This enables the CPU to perform NTT butterflies and pointwise multiplications in parallel across many elements simultaneously.

2 Performance Evaluation

In this section, we present the implementation strategies for the Rudraksh2 I parameter sets (Rudraksh2-128-I, Rudraksh2-256-I, and Rudraksh2-512-I) and provide benchmarking results.

2.1 Analysis, Considerations and Trade-offs

Implementing the NTT : Rudraksh2-128-I and Rudraksh2-256-I use the prime number 3329 as the prime modulus q . Since $q - 1 = 3328 = 2^8 \cdot 13$ is divisible by 128 and 256, full negacyclic NTT of size 64 and 128 are possible [34]. This NTT has 6 butterfly layers for $n = 64$ and 7 butterfly layers for $n = 128$. Rudraksh2-512-I uses the prime number 7681 as the prime modulus q . Since $q - 1 = 7680 = 2^9 \cdot 15$ is divisible by 512, full negacyclic NTT of size 256 is possible [34]. This NTT has 8 butterfly layers.

We implement in-place Cooley–Tukey NTT to save memory. All polynomial multiplications are done by

$$f \cdot g = \text{INTT}(\text{NTT}(f) \circ \text{NTT}(g)),$$

where “ $\circ$ ” is the pointwise product. Since both public-key and secret polynomials can be kept in the NTT domain, we minimize redundant transforms: in keygen and encapsulation, we perform forward NTT on sampled secrets once, use only pointwise multiplies (and partial sums) in the frequency domain, then do one inverse NTT per output polynomial.

We precompute the required twiddle factors (powers of ζ) and store them in memory. These fit easily in code, and no reordering is needed if we carefully index the butterflies.

Noise Sampling and Randomness : We sample secret and error polynomials from the centered binomial distribution CBD_η (e.g. $\eta = 2$). This requires generating pseudorandom bits via a stream of XOF ciphers.

2.2 Transmission and Storage Cost

Table 2 lists the key sizes (in bytes) for public-key, secret-key, ciphertext, and shared secret key for all parameter sets of Rudraksh2-I.

Table 2: Key and ciphertext sizes (in bytes) for all Rudraksh2-I parameter sets

	Public key	Secret key	Ciphertext	Shared secret key
Rudraksh2-128-I	880	1776	912	16
Rudraksh2-256-I	1760	3552	1728	32
Rudraksh2-512-I	3392	6848	3552	64

2.3 AVX2 Optimized Implementation

In this section, we provide details of our (Advanced Vector Extension) AVX2 implementation. AVX2 uses 16 vector registers, each 256 bits wide. Since we are using 16-bit primes in our implementation, we can process up to 16 16-bit integers simultaneously. Below, we have provided details of our AVX2 optimized operations:

Polynomial addition and subtraction. The polynomials for our construction have length 64, 128, and 256 for Rudraksh2-128-I, Rudraksh2-256-I, and Rudraksh2-512-I, respectively. Among the 16 vector registers, we used two registers to store 16 copies of q and the inverse of q , as they are used throughout our implementation for Montgomery multiplication, Barrett reduction, and polynomial arithmetic. Among the remaining 14 registers, we use at most 8 to store the polynomial coefficients, and the rest are used for intermediate computations. For a 64-length polynomial and a 128-length polynomial, it is enough to use 4 vector registers and 8 vector registers, respectively, to store the

complete polynomial. However, for a polynomial with length 256, we first do the computation on the first 128 coefficients, and then the rest of the 128 coefficients are processed.

NTT and inverse-NTT. We first explain our optimization method for the polynomial with length 128. The polynomial $A(x) = a_0 + a_1x + \dots + a_{127}x^{127}$ register can be easily loaded into 8 vector registers, where each register packs the polynomial $\sum_{j=0}^{15} a_{16 \cdot k + j} x^{16 \cdot k + j}$ for $k = 0, 1, \dots, 7$. In the first level of NTT, the polynomial is divided into two equal parts $A_l(x) = \sum_{i=0}^{63} a_i x^i$ and $A_h(x) = \sum_{i=64}^{127} a_i x^i$. The coefficients of A_h are multiplied by precomputed twiddle factors, and the NTT butterfly is used to update the polynomial. In the next level, the polynomials are further halved, and a similar update is applied. For the first three levels, NTT can be applied to the loaded coefficients in parallel without storing the polynomials back in the memory.

Starting at the fourth level, the sub-polynomial length drops to 16. At this stage, the standard NTT butterfly requires half of a sub-polynomial’s coefficients to be multiplied by twiddle factors while the other half remains unchanged. While we could technically use a mask of ones for the coefficients that remain unchanged, the primary issue is the subsequent butterfly addition and subtraction. AVX2 lacks a native instruction to add or subtract elements across the upper and lower halves of a single register. For polynomials with length 64, we have the same issue after the first two levels.

To overcome this limitation, we implement custom shuffling operations. These operations interleave 8, 4, or 2 elements from two different registers, isolating the coefficients to be multiplied into one register and the remaining coefficients into another. To maintain high throughput, the twiddle factors are precomputed and stored in memory in the exact interleaved order produced by this shuffling sequence. Consequently, the entire 128-point NTT is completed using only a single initial memory load, with the final results written back to memory only after the transformation is fully complete. The inverse-NTT is done by applying the same operations as in NTT, only in reverse order.

For a polynomial of length 256, loading all coefficients simultaneously would require 16 vector registers. Doing so would exhaust the register file, leaving no remaining registers available to store intermediate values during computation. We process only a 128-length segment of the polynomial at a time. Our approach relies on a key architectural observation: after the first level of the NTT, the remaining stages can be applied to the two halves of the full polynomial independently. Consequently, we can execute the first level of the NTT by splitting the workload into two memory-isolated passes: We load the first 128-coefficient subset—consisting of $(a_0, a_1, \dots, a_{63})$ and $(a_{128}, a_{129}, \dots, a_{191})$ perform the butterfly updates, and store the results back to memory. We load the remaining 128 coefficients—consisting of $(a_{64}, a_{65}, \dots, a_{127})$ and $(a_{192}, a_{193}, \dots, a_{255})$ update them, and store them back to memory. The remaining levels are completed by running the exact NTT code with different precomputed twiddle factors. In Table 3, the performance comparison of NTT of our optimized implementation against the reference implementation is given.

Table 3: Performance comparison of NTT and inverse-NTT (INTT) for all parameters of Rudraksh2-I

	Reference Code		AVX2 optimized	
	NTT	INTT	NTT	INTT
Rudraksh2-128-I	1593	1358	200	179
Rudraksh2-256-I	3203	3031	455	284
Rudraksh2-512-I	6670	5063	1013	1101

2.4 Performance of Rudraksh2

In this section, we present the performance results for reference C and optimized AVX2 implementations of Rudraksh2-I. We used a computer with an AMD Ryzen 7 7800X3D (5.053GHz x 16) processor, 32 GB RAM, running the NixOS operating system. For compilation, we used GCC version 15.2.0. According to the guidance document provided [29] the following flags were used for compilation:

- Reference: `-Wpedantic -Wall -Wextra -O2 -DNDEBUG -std=c99`
- AVX2: `-std=c99 -Wpedantic -Wall -Wextra -mavx2 -march=x86-64 -mtune=native -fomit-frame-pointer -O3 -DNDEBUG -flto=auto -fno-fat-lto-objects`

For speed benchmarking, we compiled a simple test binary with the library to measure each of the three algorithms. It used `__rdtsc` for cycle counting (Table 4, Table 7), and `clock_gettime(CLOCK_MONOTONIC, ...)` for throughput (Table 5, Table 8); taking the difference in these values before and after performing the operation. This should achieve up to nanosecond precision for the throughput measurement.

For memory benchmarking (Table 6, Table 9) we compile the algorithm into a standalone static library. Then for static measurement we do manual summation of the sizes of the respective `.text`, `.data`, `.rodata`, and `.bss` sections. For the peak usage measurement we use the `valgrind` tool `massif`. Using this we can measure the runtime memory usage, to gather both heap and stack usage we use the command:

```
valgrind --tool=massif --pages-as-heap=yes ./lwekem128-test_basic
```

Table 4: Cycle counts of the C reference implementation for all three parameter sets of Rudraksh2-I. We report the average of 1000 executions of each operation.

	Key Generation	Encapsulation	Decapsulation
Rudraksh2-128-I	2,966,367	2,947,940	2,990,398
Rudraksh2-256-I	5,760,007	5,729,075	5,816,849
Rudraksh2-512-I	18,068,660	18,029,346	18,243,608

Table 5: Throughput of the C reference implementation (in ops/s) for all three parameter sets of Rudraksh2-I. We report the average of 1000 executions of each operation.

	Key Generation	Encapsulation	Decapsulation
Rudraksh2-128-I	1,470	1,479	1,458
Rudraksh2-256-I	743	745	733
Rudraksh2-512-I	235	236	233

Table 6: Memory usage (in bytes) of the C reference implementation for all three parameter sets of Rudraksh2-I. The 'static' and 'dynamic' memory measurements were performed according to the guidance document.

	Static	Dynamic
Rudraksh2-128-I	21,471	4,608,000
Rudraksh2-256-I	22,819	4,612,096
Rudraksh2-512-I	24,571	4,612,096

In terms of execution speed, the AVX2 vectorization yields a consistent reduction in cycle counts ranging from approximately 8.8% to 14.7%, translating directly to a noticeable boost in operational throughput. For example, under the Rudraksh2-128-I parameter set, the throughput for decapsulation (detailed in Table 8) increases by 12.7%, rising from 1,458 ops/s in the reference implementation to 1,643 ops/s.

While the cycle counts in Table 7 show steady efficiency gains, the overall optimization impact varies slightly by security level. The lowest parameter set (128-I) experiences the most significant percentage increase in throughput, whereas the heaviest parameter set (512-I) sees a more modest, yet stable, speedup of around 8.1% to 9.4% across key generation, encapsulation, and decapsulation. On the memory side (as seen in Table 9), the AVX2 implementation drastically minimizes static memory footprint—cutting it by up to 82.2% for the 128-I variant. To achieve these processing efficiencies, it introduces only a marginal increase in peak dynamic memory consumption, remaining under 1.2% higher across all security tiers.

Table 7: Cycle counts of the AVX2 optimized implementation for all three parameter sets of Rudraksh2-I. We report the average of 1000 executions of each operation.

	Key Generation	Encapsulation	Decapsulation
Rudraksh2-128-I	2,569,036	2,571,855	2,549,578
Rudraksh2-256-I	5,084,174	5,081,235	5,073,256
Rudraksh2-512-I	16,350,735	16,449,056	16,464,023

Table 8: Throughput of the AVX2 optimized implementation (in ops/s) for all three parameter sets of Rudraksh2-I. We report the average of 1000 executions of each operation.

	Key Generation	Encapsulation	Decapsulation
Rudraksh2-128-I	1,635	1,630	1,643
Rudraksh2-256-I	825	825	826
Rudraksh2-512-I	256	255	255

Table 9: Memory usage (in bytes) of the AVX2 optimized implementation for all three parameter sets of Rudraksh2-I. The 'static' and 'dynamic' memory measurements were performed according to the guidance document.

	Static	Peak
Rudraksh2-128-I	3,816	4,653,056
Rudraksh2-256-I	8,256	4,653,056
Rudraksh2-512-I	18,456	4,665,344

2.5 Performance of Rudraksh2 on Cortex-M4

The Rudraksh2 schemes were implemented on the NIST-recommended 32-bit ARM Cortex-M4 platform using the STM32F407G-DISC1 development board within the PQM4 framework [22]. All implementations were cross-compiled on Ubuntu 22.04.5 with `arm-none-eabi-gcc` version 10.3.1 (`-O3` optimization flag). Following the standard PQM4 benchmarking methodology, clock-cycle measurements were obtained using a system clock frequency of 24 MHz. The implementation results for the Rudraksh2 schemes on the Cortex-M4 platform are presented in Tables 11 and 10. These optimized Cortex-M4 implementations are constant-time, which means they avoid any control-flow decisions or instructions whose execution depends on secret data, such as secret-dependent branches, jumps, or variable-time operations. Consequently, their execution time is independent of sensitive inputs, making them resilient to timing side-channel attacks.

Constant-time Arithmetic. We follow the reference implementation to incorporate constant-time modular reduction using Montgomery multiplication and Barrett reduction [8].

Table 10: Speed (x1000 clock cycles) of the optimized implementations for all Rudraksh2-I parameter sets on Cortex-M4

	Key Generation	Encapsulation	Decapsulation
Rudraksh2-128-I	9,960	9,948	10,124
Rudraksh2-256-I	19,837	19,903	20,296
Rudraksh2-512-I	64,023	63,959	65,097

2.5.1 Memory Optimization on Cortex-M4

To minimize stack usage on Cortex-M4 platforms, the implementation has been restructured to avoid placing large intermediate data structures on the stack. Instead of explicitly forming complete matrices,

vectors, or intermediate ciphertext objects, the computations are carried out in a streaming fashion whenever possible. We follow some of the optimization techniques from Botros et al. [13].

Just-in-Time Matrix Generation. The public matrix $\mathbf{A}$ is formed dynamically from its seed, generating one polynomial at a time, which eliminates the need for full storage. Similarly, intermediate results of polynomial-vector multiplications are not stored as complete entities.

In-line Ciphertext Comparison. During decapsulation, we implemented the in-line ciphertext comparison technique to integrate the verification step directly into the computation of the ciphertext, avoiding the need to materialize an intermediate ciphertext buffer. Instead of storing and then comparing full ciphertexts, the freshly generated ciphertext is compared byte-by-byte against the received ciphertext during computation. This approach reduces memory overhead while preserving constant-time execution, thereby maintaining resistance to timing side-channel attacks.

On-the-fly Noise Addition. Instead of storing the error vectors $\mathbf{e}$ and $\mathbf{e}'$ as complete polynomial-vector objects, which incurs a higher stack memory overhead, the implementation generates each error polynomial on demand and incorporates it immediately into the computation. This polynomial-by-polynomial approach eliminates the need to maintain the full error vectors in memory, thereby significantly reducing stack usage.

Reduced Vector Variables. To further reduce stack memory usage during key encapsulation, we introduce serialization and deserialization routines that process polynomial vectors one polynomial at a time. Rather than compressing or decompressing an entire polynomial vector in memory, these routines incrementally convert between the polynomial-vector representation and its serialized byte array format. This streaming approach minimizes the number of polynomial-vector variables that must be allocated on the stack, thereby lowering the overall memory footprint.

Table 11: Stack usage (in bytes) for the optimized implementations for all Rudraksh2-I parameter sets on Cortex-M4

	Key Generation	Encapsulation	Decapsulation
Rudraksh2-128-I	4,848	4,920	4,936
Rudraksh2-256-I	7,976	8,088	8,088
Rudraksh2-512-I	14,344	14,520	14,488

This design introduces a modest amount of recomputation and temporary buffering but substantially reduces stack requirements, making it well-suited for resource-constrained embedded platforms such as the Cortex-M4. A summary of stack usage across all Cortex-M4 implementations is provided in Table 11.

2.6 Hardware Design

Based on theoretical design decisions, the hardware implementation is focused on developing a lightweight lattice-based KEM. The target devices will be Artix-7 and Virtex-7 FPGAs. To validate the theoretical framework for the resource-constrained design, a unified hardware architecture is proposed.

2.6.1 Proposed System Architecture

The proposed architecture is shown in Figure 1. A unified FSM-based top module is proposed to coordinate the execution of the core functions. In addition to scheduling the execution of the KeyGen, Encaps, and Decaps operations based on the input control signals, the centralized FSM manages data movement between the individual modules and the BRAMs. While the NTT/INTT and the coefficient-wise multiplication (CWM) module internally uses two BRAMs for performing the butterfly operations, a separate BRAM in the top module will be interfaced with remaining modules to store other intermediate results, along with the final public key, secret key and ciphertext. The SM3 hashing

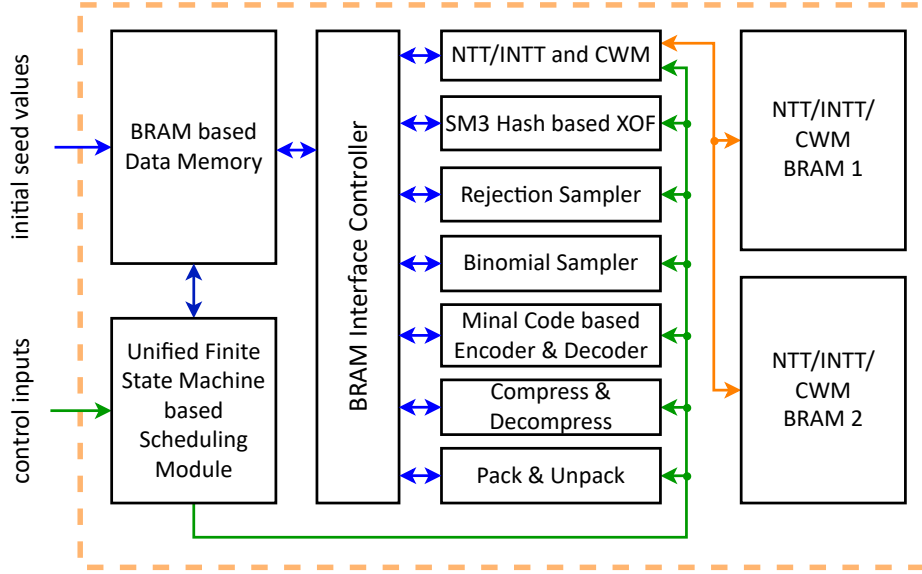


Figure 1: Proposed Top Level Hardware Architecture for Rudraksh2

standard is used for the generation of hashes and pseudo-randomness based on the input seed values. It serves as the key module used for the generation of matrix $\hat{\mathbf{A}}$. The rejection sampler is required to discard coefficients greater than or equal to the prime modulus q since each coefficient of the matrix should be $\lceil \log_2 q \rceil$ bits and less than q . The SM3 core, extended as a pseudoXOF, is also used to generate the pseudo-random numbers for the CBD sampler for generating the secret and error vectors, $\mathbf{s}$ and $\mathbf{e}$. Once the polynomial coefficients are generated, a hardware-friendly butterfly unit performs NTT/INTT and CWM operations. Although the reference software implementation of Rudraksh2 uses Montgomery and Barrett reductions for modular reduction, our hardware adopts a different approach wherein simpler shift and add operations are used, making it suitable for resource-constrained devices. To achieve this, a shift and add modular reduction method proposed in [25] is used for $q = 7681$, and a combination of LUT-based reduction and k -reduction proposed in [10] is utilized for modulus $q = 3329$. It should be noted that [25] implements $q = 7681$ in a pipelined fashion; however, a combinational implementation of $q = 3329$ resulted in a significantly lower area at the cost of a reduced critical path. Shift-and-add based technique for reduction is extended from [20]. While the resource utilization results are presented for these reduction techniques, further explorations are being done to evaluate alternative architectures to find the one most suited to a resource-constrained implementation.

2.6.2 SM3 as a XOF

The proposed pseudoXOF is realized as an FSM-controlled wrapper around a single SM3 hash core. Since SM3 produces a fixed 256-bit digest, the wrapper repeatedly invokes the hash core to generate arbitrary-length outputs. A 32-bit counter is initialized to one and appended to the input message before each hash invocation. The resulting 256-bit digest is written to the output buffer, after which the counter is incremented and the process is repeated until the requested output length is generated. Once each digest is valid, it is also stored through the BRAM interface into the BRAM data memory.

The wrapper comprises a finite state machine, a counter register, input/output buffers, and the SM3 core. The FSM coordinates message preparation, hash execution, output accumulation, and termination, while reusing a single SM3 instance across all iterations. This approach incurs only a small control overhead while enabling extendable outputs of arbitrary length.

The SM3 hash core, including its compression and padding logic, is adopted from an existing open-source implementation [32] with minor interface modifications for integration into the proposed wrapper. As the contribution of this work lies in the hardware realization of the pseudoXOF wrapper, the internal architecture of the SM3 core is not discussed further.

2.6.3 Resource Utilization

The resource utilization results of the arithmetic data path as well as PseudoXOF, on a target Virtex UltraScale+ device, are provided in Table 12.

Table 12: Resource Utilization of individual modules on a target Virtex UltraScale+ device

Modulus	Module	Resource Utilization			
		LUT	FF	DSP	CARRY8
$q = 7681$	Butterfly	311	299	1	12
	Reduction	85	0	0	2
	CBD	5	0	0	0
$q = 3329$	Butterfly	151	0	1	17
	Reduction	49	0	0	14
	CBD	5	0	0	0
	Rejection sampling	485	81	0	25

Table 13: Resource Utilization of SM3 PseudoXOF Module on a target Virtex UltraScale+ device

Module	Resource Utilization			
	LUT	FF	DSP	CARRY8
SM3 XOF Wrapper	2256	2155	0	34
SM3 Core [32]	2213	1856	0	27
Compression Function	1004	637	0	12
Message Expansion	99	578	0	0
Message Padding	1101	628	0	15

3 Security Analysis

Rudraksh2.KEM, in short, Rudraksh2, is an IND-CCA2 key-encapsulation mechanism. The classical core-SVP hardness (CSVP), quantum core-SVP hardness (QSVP), and failure probability (δ) for different parameter sets of Rudraksh2 are presented in Table 14. Beyond Core-SVP hardness security, the estimated cost of classical attacks using [3] in terms of gates and bit memory requirements is also presented in Table 14.

Table 14: Classical core-SVP hardness (CSVP), quantum core-SVP hardness (QSVP), and failure probability (δ) for different parameter sets of Rudraksh2. The estimated cost of classical attacks using [3] is given in terms of gates and bit memory requirements.

	CSVP	QSVP	δ	Gates	Bit memory
Rudraksh2-128-I	2^{129}	2^{117}	2^{-100}	2^{162}	2^{101}
Rudraksh2-128-II	2^{114}	2^{103}	2^{-100}	2^{148}	2^{91}
Rudraksh2-256-I	2^{267}	2^{242}	2^{-161}	2^{303}	2^{201}
Rudraksh2-256-II	2^{261}	2^{237}	2^{-167}	2^{297}	2^{197}
Rudraksh2-512-I	2^{512}	2^{465}	2^{-160}	2^{527}	2^{360}
Rudraksh2-512-II	2^{589}	2^{534}	2^{-160}	2^{587}	2^{403}

3.1 Theoretical Security

Underlying hard problem. The security of Rudraksh2 is based on the hardness of the module learning with errors (MLWE) problem.

Given $\mathbf{A} \leftarrow \mathcal{U}(R_{q,n}^{\ell \times \ell})$, $\mathbf{s} \leftarrow \beta_{\eta_1}(R_{q,n}^\ell)$, $\mathbf{e} \leftarrow \beta_{\eta_2}(R_{q,n}^\ell)$, and $\mathbf{b} \leftarrow \mathcal{U}(R_{q,n}^\ell)$. The MLWE problem states that the MLWE instance is $(\mathbf{A}, b = \mathbf{A}\mathbf{s} + \mathbf{e}) \in R_{q,n}^{\ell \times \ell} \times R_{q,n}^\ell$ is indistinguishable with $(\mathbf{A}, \mathbf{b})$.

Advantage of a MLWE-based algorithm $\mathcal{A}$ with parameters q, n, ℓ and distribution of secret and error η_1, η_2 is defined by $\text{Adv}_{q,n,\ell,\ell_1,\eta_1,\eta_2}^{\text{MLWE}}(\mathcal{A}) =$

$$\left| \Pr \left(\begin{array}{l} \mathbf{A} \leftarrow \mathcal{U}(R_{q,n}^{\ell_1 \times \ell}); \mathbf{s} \leftarrow \beta_{\eta_1}(R_{q,n}^\ell); \mathbf{e} \leftarrow \beta_{\eta_1}(R_{q,n}^{\ell_1}) \\ b' = 1 : b := \mathbf{A}\mathbf{s} + \mathbf{e} \\ b' := A(\mathbf{A}, b) \end{array} \right) - \Pr \left(\begin{array}{l} \mathbf{A} \leftarrow \mathcal{U}(R_{q,n}^{\ell_1 \times \ell}) \\ b' = 1 : \mathbf{b} \leftarrow \mathcal{U}(R_{q,n}^{\ell_1}) \\ b' := A(\mathbf{A}, \mathbf{u}) \end{array} \right) \right|$$

3.1.1 IND-CCA security proof under the classical random oracle model

Based on the hardness assumption of MLWE problem [1], Rudraksh2.PKE is IND-CPA secure.

Theorem 1. For an IND-CPA adversary $\mathcal{A}$, there exist two adversaries $\mathcal{B}_0$ and $\mathcal{B}_1$ such that

$$\text{Adv}_{\text{Rudraksh2.PKE}}^{\text{IND-CPA}}(\mathcal{A}) \leq 2\text{Adv}_{q,n,\ell,(\ell+1),\eta_1,\eta_2}^{\text{MLWE}}(\mathcal{B}_0) + \text{Adv}_{\text{PRF}}^{\text{prf}}(\mathcal{B}_1).$$

The above theorem has been proven in [7, 12].

Correctness: If Rudraksh2.PKE is $(1 - \delta)$ -correct then Rudraksh2.KEM is also $(1 - \delta)$ -correct [21].

Theorem 2. Given an IND-CCA adversary $\mathcal{A}$ can query the random oracle XOF, G and H at most q_{RO} times, there exist two adversaries $\mathcal{B}_0$ and $\mathcal{B}_1$ such that the following relation holds:

$$\text{Adv}_{\text{Rudraksh2.KEM}}^{\text{IND-CCA}}(\mathcal{A}) \leq 2\text{Adv}_{q,n,\ell,(\ell+1),\eta_1,\eta_2}^{\text{MLWE}}(\mathcal{B}_0) + \text{Adv}_{\text{PRF}}^{\text{prf}}(\mathcal{B}_1) + 4q_{\text{RO}}\delta$$

The above theorem has been proven in [7, 21].

3.1.2 IND-CCA security proof under the quantum random oracle model

Theorem 3. *Given an quantum IND-CCA adversary $\mathcal{A}$ can query the random oracle XOF , G and H at most q_{RO} times, there exist two quantum adversaries $\mathcal{B}_0$ and $\mathcal{B}_1$ such that the following relation holds:*

$$Adv_{Rudraksh2.KEM}^{IND-CCA}(\mathcal{A}) \leq 4q_{RO} \sqrt{Adv_{q,n,\ell,(\ell+1),\eta_1,\eta_2}^{MLWE}(\mathcal{B}_0) + Adv_{PRF}^{prf}(\mathcal{B}_1) + 8q_{RO}^2 \delta}$$

The above bound is proposed in [21], but it is not a tight bound. A tight bound is proposed in [33]. For this, we need a deterministic version of Rudraksh2.PKE, DRudraksh2.PKE is a pseudorandom function under the quantum random oracle model.

Theorem 4. *Given an quantum IND-CCA adversary $\mathcal{A}$ can query the random oracle XOF , G and H at most q_{RO} times, there exist two quantum adversaries $\mathcal{B}_0$, $\mathcal{B}_1$, and $\mathcal{B}_2$ such that the following relation holds:*

$$Adv_{Rudraksh2.KEM}^{IND-CCA}(\mathcal{A}) \leq 2Adv_{q,n,\ell,(\ell+1),\eta_1,\eta_2}^{MLWE}(\mathcal{B}_0) + Adv_{DRudraksh2.PKE}^{IND-CPA}(\mathcal{B}_1) + Adv_{PRF}^{prf}(\mathcal{B}_2) + 8q_{RO}^2 \delta$$

3.2 Practical Security

This section presents a comprehensive summary of the practical security of our KEM scheme, Scabbard.

3.2.1 Attacks against LWE

Rudraksh is an MLWE-based KEM. As there are no known attacks that effectively exploit the module lattice structure, we will treat MLWE security as standard LWE security. In our KEM, we have only $m = (\ell + 1)n$ samples available; we exclude some of the existing LWE solvers [2] for our parameter set. BKW [24] and linearization attacks [6] are those types of attacks we are not bothering with in our case. In our KEM, we are only focused on BKZ attacks [14, 35], which are known as Primal and Dual BKZ-based attacks.

The BKZ algorithm reduces a lattice basis by repeatedly invoking an SVP oracle on blocks of dimension b . In paper [5], the authors introduce the hardness of the core-SVP methodology. Using this methodology, we can calculate the computational cost of a single SVP oracle execution in dimension b . BKZ depends on two main approaches for its SVP oracle: enumeration and sieving algorithms. Because these methods scale differently, it is difficult to accurately predict how sieving will perform at the large lattice dimensions required to attack schemes like Scabbard. Theoretically, enumeration runs in super-exponential time while sieving is strictly exponential, which suggests sieving scales better. However, in practice, enumeration is faster and takes less memory for smaller dimensions [14, 19]. For our security analysis, we employ $2^{0.292b}$ for classical costs and $2^{0.265b}$ for quantum costs in both primal and dual attacks. We explain both the attack costs as follows:

3.2.2 Primal attack

The primal attack is used to evaluate the cost of the security of the Learning With Errors (LWE) problem. Rather than attacking the LWE equations directly, it transforms the LWE instance into a specific lattice challenge, the Unique Shortest Vector Problem (uSVP). The primary target of this analysis is to determine the minimum BKZ block dimension (b) required to succeed, as this block size directly dictates the computational time and cost of the attack.

To do the attack for given LWE instance $(\mathbf{A}, \mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e})$, we first construct the following lattice: The lattice is defined as the set of vectors x satisfying $\Lambda = \{\mathbf{x} \in \mathbb{Z}^{\ell m + \ell n + 1} : (\mathbf{A}|\mathbf{I}_\ell) \cdot \mathbf{x} = \mathbf{b} \mod q\}$. This newly formed lattice has a total dimension of $d = \ell n + m + 1$ and a mathematical volume of $\text{Vol}(\Lambda) = q^m$. By construction, this lattice contains a short vector: $\mathbf{v} = (\mathbf{s}, \mathbf{e}, 1)$. The expected norm of this vector is $\lambda \approx \sigma \sqrt{\ell n + m}$. Here, σ represents the standard deviation of the individual coefficients within both the secret key ($\mathbf{s}$) and the error ($\mathbf{e}$). The attacker can choose any number (between 0 to $n\ell + n$) of the LWE samples. Since the choice of ℓ changes both the dimension (d) and the volume (q^ℓ) of the lattice, we can select ℓ according our choice to find the exact "sweet spot" that minimizes the required BKZ block size.

To mathematically predict whether the attack will work, we use Geometric Series Assumption (GSA) which finds the Gram-Schmidt orthogonalized basis vectors having norm $\|\mathbf{b}_i^*\| = \delta^{d-2i-1}$. $\text{Vol}(\Lambda)^{\frac{1}{d}}$ where, $\delta = \left((\pi b)^{\frac{1}{b}} \cdot \frac{b}{2\pi e}\right)^{\frac{1}{2(b-1)}}$, is the root Hermite factor [2].

The BKZ algorithm will successfully detect the unique short vector $\mathbf{v}$ if the projection of $\mathbf{v}$ onto the vector space spanned by the last b Gram-Schmidt vectors is $\leq \|b_{d-b}^*\|$. The expected norm of $\mathbf{v}$'s projection is expected to be $\sigma\sqrt{b}$. Therefore, the attack succeeds if and only if: $\sigma\sqrt{b} \leq \|b_{d-b}^*\|$ i.e., iff $\sigma\sqrt{b} \leq \delta^{2b-d-1} \cdot q^{\frac{m}{d}}$.

3.2.3 Dual attack

The dual attack targets to distinguish LWE samples $(\mathbf{A}, \mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e})$ from random values $(\mathbf{A}, \mathbf{b}')$ by finding a short vector in the dual lattice $\Lambda' = \{(\mathbf{x}, \mathbf{y}) \in \mathbb{Z}^m \times \mathbb{Z}^{\ell n} : \mathbf{A}^\top \mathbf{x} = \mathbf{y} \bmod q\}$. Suppose the shortest vector is $\mathbf{u} = (\mathbf{v}, \mathbf{w}) \in \mathbb{Z}^m \times \mathbb{Z}^{\ell n}$ in the lattice Λ' . Now, we can compute a value $z = \mathbf{v}^\top \cdot \mathbf{b} = \mathbf{v}^\top \cdot (\mathbf{A}\mathbf{s} + \mathbf{e}) = \mathbf{v}^\top \cdot \mathbf{A}\mathbf{s} + \mathbf{v}^\top \cdot \mathbf{e} = \mathbf{w}^\top \mathbf{s} + \mathbf{v}^\top \mathbf{e} \bmod q$. If $(\mathbf{A}, \mathbf{b})$ is a valid LWE sample, z follows a Gaussian distribution with standard deviation $\sigma' = \|\mathbf{u}\|\sigma$; if it is random, z is uniform. The attacker's advantage to distinguish these two distributions is $\epsilon = 4 \exp(-2\pi^2 \tau^2)$, where $\tau = \frac{\|\mathbf{u}\|\sigma}{q}$. The BKZ algorithm provides a short vector of length $\approx \delta^{d-1} q^{\ell n/d}$, where $d = m + \ell n$ is the dimension of the dual lattice and δ is the root-Hermite factor associated with the block size b . To obtain a distinguisher with advantage ϵ , the attacker must run BKZ with a block dimension b that satisfies: $-2\pi^2 \tau^2 \geq \ln(\epsilon/4)$. Since the shared key is hashed, a negligible advantage ϵ is useless; the attacker requires an advantage of at least $\frac{1}{2}$ to effectively reduce the search space. To achieve this, the attacker must amplify their success probability by collecting multiple short vectors. If $N = 2^{0.2075b}$ is the number of short vectors provided by a sieving algorithm, the number of repetitions R required is: $R = \max(1, \frac{1}{N \cdot \epsilon^2})$. This conservative model assumes every generated vector is as short as the best one found. By combining them, the attacker reaches the threshold needed to recover the secret key.

3.2.4 Core-SVP hardness

Core-SVP hardness is used to estimate the security of LWE-based schemes, which is presented in [4]. We use Dachman-Soled et al.'s leaky-LWE-Estimator [15] to estimate SVP hardness of the underlying LWE-problem of our scheme. Classical Core-SVP security is $0.292 * \beta$, and quantum Core-SVP security is $0.265 * \beta$, where β is the BKZ blocksize of the underlying LWE problem. We presented classical and quantum core-SVP security of the parameter sets of Rudraksh2 in Table 14.

3.2.5 Beyond core-SVP hardness

The Core-SVP methodology used in the last subsections provide a standard, conservative framework for quantifying the security of Rudraksh2. However, it is important to note that these bit-security estimates are strictly lower bounds. In practice, the security of Rudraksh2 against lattice reduction attacks is significantly higher due to factors not captured by the Core-SVP metric:

- **BKZ Algorithm Overhead** : Core-SVP assumes the cost of an attack is dominated by a single call to an SVP oracle in dimension b . In reality, the Block-Korkine-Zolotarev (BKZ) algorithm requires a large number of calls to the SVP oracle to converge. Furthermore, the preprocessing steps and the basis reduction process itself introduce substantial computational overhead that increases the total work factor for the attacker.
- **Memory-Hardness of Sieving** : The most efficient known algorithms for solving the SVP, such as sieving, are memory-intensive. The bit-complexity formulas often assume an infinite-memory model. In physical hardware, the requirement for massive amounts of memory to store and process large lists of lattice vectors creates a practical bottleneck that restricts the efficiency of parallelization. This "memory-wall" makes practical sieving attacks substantially more costly than a simple count of operations suggests.
- **Quantum Sieving Costs**: Our quantum security estimates ($2^{0.265b}$) are based on the assumption that Grover's algorithm can provide a quadratic speedup for the SVP oracle. However, the overhead of implementing a full sieving algorithm within a quantum circuit has massive quantum memory requirements. This remains a major implementation hurdle. If the quantum memory is not free (i.e., if it has an area/latency cost), the effective security margin of Rudraksh2 increases significantly.

An estimate of Rudraksh2’s security (gate count) and bit memory requirements considering these constraints has been shown in Table 14. This analysis follows the same procedure described in Sec. 5.2 of the Kyber Specification document [7] and resource estimations shown in [3]. Given that our Core-SVP estimates for Rudraksh2 already meet or exceed the required security, these practical overheads in security calculations provide a significant “security buffer”. This ensures that Rudraksh2 remains secure even against future improvements in lattice reduction heuristics or quantum-algorithmic breakthroughs.

3.2.6 Attacks using decryption failures

Like most lattice-based encryption schemes based on the Learning with Errors (LWE) or Module Learning with Errors (MLWE) problem, the underlying public-key encryption scheme of Rudraksh2 (Rudraksh2.PKE) Algorithm 10, 11, 12 does not possess perfectly correct decryption. A decryption failure occurs when the cumulative noise originating from the Centered Binomial Distribution (CBD) error terms, secret values, and the rounding noise introduced by ciphertext compression grows large enough to cross the tolerance of the underlying error-correcting mechanism.

The Attack Vector: In a Chosen-Ciphertext Attack (CCA), an adversary could theoretically exploit these decryption failures. If an attacker can find valid ciphertexts that fail to decrypt, the failure events leak information about the honest party’s secret key $\mathbf{s}$. By collecting enough of these failing ciphertexts, the attacker can systematically reconstruct the secret key (often referred to as a Decryption Failure Attack or Failure Boosting). While the Fujisaki-Okamoto (FO) transform is utilized in Rudraksh2.KEM prevents an attacker from simply querying maliciously malformed ciphertexts; the attacker can still mount a “failure boosting” attack by searching offline for valid ciphertexts (and their corresponding random coins) that have an unusually high probability of failing.

Security Justification: To mount this attack successfully, the adversary must find a sufficient number of failing ciphertexts. The expected offline work required to find a single naturally failing ciphertext is proportional to $1/\delta$, where δ is the decryption failure probability. The parameters of Rudraksh2, combined with the use of extended B2-Minal codes for error correction, have been carefully optimized to ensure that δ is cryptographically negligible. According to our failure probability analysis (summarized in Table 1):

- For the Level 1 parameters, the failure probability is $\delta \leq 2^{-100}$ for both Rudraksh2-128-I and Rudraksh2-128-II.
- For the Level 3 parameters, the failure probability is $\delta \leq 2^{-161}$ for Rudraksh2-256-I and $\delta \leq 2^{-167}$ for Rudraksh2-256-II.
- For the Level 5 parameters, the failure probability is $\delta \leq 2^{-160}$ for both Rudraksh2-512-I and Rudraksh2-512-II.

In the usual assumption of the IND-CCA2 security model, an adversary is restricted to at most $q \leq 2^{64}$ online queries to the decapsulation oracle [28, 7]. For our highest parameter sets, the probability of observing even a single failure within this limit is very small (e.g., $2^{64} \times 2^{-160} = 2^{-96}$). Furthermore, a successful key-recovery attack requires the collection of multiple decryption failures. While an attacker may attempt offline ‘failure boosting’ to find candidate ciphertexts, the computational cost of precomputing enough weak ciphertexts to force multiple online failures strictly exceeds the target security levels of the scheme. Therefore, attacks exploiting decryption failures provide no practical threat to Rudraksh2.

3.3 Additional Security Properties

3.3.1 (Perfect) Forward Secrecy

Rudraksh2 is a Key-Encapsulation Mechanism (KEM) designed to securely transport symmetric session keys. As with other lattice-based KEMs (e.g., ML-KEM/Kyber), Rudraksh2 does not provide forward secrecy as an inherent property of the KEM algorithm itself, but rather as an architectural property of the protocol in which it is deployed.

To achieve (perfect) forward secrecy using Rudraksh2, protocol designers must ensure that the KEM is used in an ephemeral mode. This requires the following:

- **Ephemeral Key Generation:** For every new communication session, both parties must generate a fresh, ephemeral Rudraksh2 key pair.
- **Secure Deletion:** Once the shared secret has been encapsulated/decapsulated and the session key has been derived, the ephemeral Rudraksh2 secret key must be securely deleted (zeroized) from memory.
- **Protocol Requirement:** The protocol must ensure that long-term keys are kept separate from the ephemeral KEM keys. Even if the long-term keys are compromised in the future, an adversary cannot recover past session keys, as the corresponding ephemeral Rudraksh2 private keys were discarded at the end of the session.

When deployed in this manner within a secure framework such as an authenticated key agreement protocol, Rudraksh2 provides the foundation for forward secure communication, ensuring that past sessions remain protected against the compromise of secret credentials.

3.3.2 Side-Channel Attack Resilience:

All operations in Rudraksh2, including secret polynomial sampling, NTT transformations, seed expansion, and the symmetric hash functions, are implemented in constant time. The implementation specifically avoids data-dependent branching, e.g., if-then-else statements dependent on secret key bits and data-dependent memory access patterns, e.g., secret-dependent array indexing, which are often primary vectors for timing and cache-based side-channel attacks. Rudraksh2 employs Minal codes to reduce the decryption failure probability. Unlike generic decoding algorithms, which may have variable execution times depending on the number of errors, our Minal code implementation is engineered to be strictly constant-time. This ensures that the decoding process does not leak information about the noise weight or the proximity to the decoding boundary, thereby thwarting timing-based failure attacks. These designs make our implementation of Rudraksh2 protected from timing and simple power analysis attacks.

While the algorithmic foundation is constant-time, real-world physical implementations may still be vulnerable to Differential Power Analysis (DPA) or correlation power analysis (CPA). However, such attacks are possible for any cryptographic schemes that use non-ephemeral keys. To protect a cryptographic scheme from such attacks, dedicated defense mechanisms such as masking or shuffling can be employed.

3.3.3 Multi-key Attack Resilience

The security of Rudraksh2 is evaluated against an adversary who may target one out of T public keys, a setting known as the multi-target or multi-user security model. In this setting, the adversary's advantage in breaking the KEM is defined by their ability to successfully encapsulate/decapsulate a key for any of the T observed targets.

Public-key binding in Encaps operation: To ensure that the security of Rudraksh2 does not degrade significantly as the number of users T increases, we enforce strict domain separation through public key binding. During the encapsulation process, the random coins r used for the deterministic encryption of the message m are generated as:

$$(K, seed_r) = \mathcal{G}(\mathcal{H}(pk) \parallel m)$$

. By hashing the public key pk and including it in the derivation of the encryption coins $seed_r$, we ensure that the encryption process for every public key is cryptographically independent.

Multi-Target Security Bound: Let $\text{Adv}_T^{\text{IND-CCA}}$ be the advantage of an adversary in breaking the IND-CCA2 security of Rudraksh2 in a setting with T distinct public keys. Under the Random Oracle Model (ROM), we relate this to the security of a single target:

$$\text{Adv}_T^{\text{IND-CCA}} \approx T \cdot \text{Adv}_1^{\text{IND-CCA}}$$

This linear degradation is the best achievable bound in the standard KEM setting. Because we bind pk to the internal coin generation and key derivation, the adversary cannot amortize the cost of the

attack (e.g., via precomputed lattice reduction or rainbow tables) across different targets. Each target effectively acts as an independent random oracle instance.

Comparison to Generic Multi-Key Bounds: In generic constructions, the advantage of breaking a Pseudo-Random Function (PRF) or a KEM often degrades by a factor related to the number of evaluations or keys queried [9]. In Rudraksh2, due to the binding of $\mathcal{H}(pk)$ at the point of generating the ephemeral coins $seed_r$ and the pre-shared key K , the adversarial advantage is constrained as follows:

- **No Amortization :** An adversary cannot use ciphertexts intended for one target to gain information about another, as the re-encryption check in the FO transform is explicitly bound to the public key.
- **Deterministic Generation :** The deterministic expansion of the matrix $\mathbf{A}$ from a seed, combined with the public key binding, ensures that the lattice instance itself is unique to each user. Even in a quantum setting, where Grover’s search might be applied, the uniqueness of the key-bound hash prevents the adversary from parallelizing the attack effectively across multiple users.

Consequently, Rudraksh2 achieves tight multi-target security bounds. The cost to compromise any one of the T users remains proportional to the Core-SVP hardness of the underlying MLWE instance, ensuring that widespread deployment in global IoT ecosystems does not mathematically dilute the security of the scheme.

3.3.4 (Temporary) Key Misuse Resilience

In real-world deployments, cryptographic implementations are frequently subjected to operational misuse, such as random number generator (RNG) failures, key reuse, or protocol-level identity confusion. Rudraksh2 is designed to minimize the security impact of such failures by incorporating several security protocols directly into the key-derivation, encapsulation, and decapsulation phases.

Resilience against weak randomness (RNG Failures): The security of any public-key system relies heavily on the quality of the system’s Random Number Generator (RNG). Rudraksh2 isolates the impact of RNG failures depending on the phase of the operation:

- **During Encapsulation:** If the system RNG produces weak or predictable randomness (m) during encapsulation, the adversary can only compromise that specific session key. Because the encryption randomness (the coins r) is derived deterministically via $r = \mathcal{G}(\mathcal{H}(pk) \parallel m)$, which in turn is used to generate session secret s' and e' . Therefore, a weak RNG during encapsulation can compromise the session keys, but it cannot be exploited to extract the receiver’s long-term secret key.
- **During Decapsulation:** The decapsulation algorithm of Rudraksh2 requires no fresh randomness. To handle invalid ciphertexts securely, the decapsulation algorithm relies on the implicit rejection provided by the FO transformation. A 32-byte pseudo-random string z is generated during **KeyGen** and stored within the secret key. If a ciphertext fails the re-encryption check, the decapsulation deterministically derives a rejection key using z and the ciphertext. This guarantees that the KEM never outputs a predictable error code, preventing timing or padding-oracle attacks even in poorly implemented protocols.

Public-key binding: To prevent protocol-level misuse, Rudraksh2 cryptographically binds the encapsulation process to the recipient’s exact public key. In `Rudraksh2.Encaps`(Algorithm. 14), the shared key and the encryption coins are derived by hashing the random message m alongside the hash of the public key:

$$(K, seed_r) = \mathcal{G}(\mathcal{H}(pk) \parallel m)$$

By injecting $\mathcal{H}(pk)$ into the derivation, the scheme achieves strict domain separation. If a protocol misuses the KEM—such as a developer accidentally encrypting the same message m to multiple recipients, or an attacker attempting to replay a ciphertext across different users, this derivation process ensures the resulting ciphertexts and internal states are completely uncorrelated. This also guarantees that a weakness against one user cannot be leveraged to compromise a broader network of users.

API misuse prevention: To account for developer error in mixing up keys for different identities, the secret key in Rudraksh2 is constructed as a composition of $sk = (s \parallel pk \parallel \mathcal{H}(pk) \parallel z)$. By caching the public key and its hash directly within the secret key array, the decapsulation routine has safe, zero-cost access to pk for the required re-encryption step. This prevents API misuse, as developers are not required to manually manage or pass the public key to the decapsulation function, eliminating a common source of protocol integration errors.

4 Advantages and Limitations

Rudraksh has been designed to fill a specific gap in the post-quantum landscape: providing IND-CCA2 security even in platforms with very low resources. The following subsections compare Rudraksh against NIST-standardized schemes and other prominent candidates, highlighting its unique advantages for IoT and resource-constrained systems.

Comparison to CRYSTALS-Kyber. As both schemes are based on the Module-LWE (MLWE) problem, they share a common mathematical foundation. However, Rudraksh departs from Kyber’s fixed-ring design to achieve superior efficiency on small hardware and also greater flexibility. Rudraksh has been designed to be significantly more compact than the most resource-optimized versions of Kyber. Preliminary implementation results show that Rudraksh achieves an approximately $3\times$ reduction in total area and memory compared to resource-optimized Kyber. Specifically, in terms of Equivalent Number of Slices (ENS), Rudraksh consumes $4.3\times$ less ENS than compact Kyber implementations. However, for communication bandwidth, Rudraksh makes some compromise. To achieve its extreme area efficiency, Rudraksh has slightly larger public keys. This is a deliberate design trade-off that prioritizes hardware resource conservation over bandwidth.

Comparison to SABER. While SABER utilizes the Learning with Rounding (LWR) problem to avoid explicit error sampling, Rudraksh offers better flexibility. SABER utilizes power-of-two moduli, which prevents the use of the Number Theoretic Transform (NTT). Instead, it relies on generic multiplication algorithms (Toom-Cook and Karatsuba), which require significant temporary memory for intermediate results. Rudraksh’s NTT-based multiplication is in-place and supports “Lazy Transformation,” where inverse transforms are deferred to save cycles and energy. In software (e.g., ARM Cortex-M4), SABER’s larger polynomials leads to higher stack memory requirements compared to the small polynomial rings ($n = 64, 128$) used in Rudraksh.

FrodoKEM. FrodoKEM is based on standard lattices and therefore lacks algebraic ring structure, resulting in much larger keys and expensive matrix-vector multiplications. Rudraksh offers much smaller key sizes and efficient arithmetic compared to FrodoKEM.

NTRU-HRSS and NTRU. Compared to these classic lattice designs, Rudraksh is vastly more resource-efficient. Initial implementations show that Rudraksh requires $23.9\times$ less ENS than NTRU-HRSS701 and $21.5\times$ less ENS than NTRU-HPS677 while maintaining equivalent security margins.

References

- [1] D. Stehlé A. Langlois. Worst-case to average-case reductions for module lattices. *Des. Codes Cryptogr.* 75, 2015.
- [2] Martin Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. *Journal of Mathematical Cryptology*, 9, 10 2015.
- [3] Martin R. Albrecht, Vlad Gheorghiu, Eamonn W. Postlethwaite, and John M. Schanck. Estimating quantum speedups for lattice sieves. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology - ASIACRYPT 2020 - 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7-11, 2020, Proceedings, Part II*, volume 12492 of *Lecture Notes in Computer Science*, pages 583–613. Springer, 2020.
- [4] Erdem Alkim, Léo Lucas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum key exchange - A new hope. In Thorsten Holz and Stefan Savage, editors, *25th USENIX Security Symposium, USENIX Security 16, Austin, TX, USA, August 10-12, 2016*, pages 327–343. USENIX Association, 2016.
- [5] Erdem Alkim, Léo Lucas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum key exchange: a new hope. In *Proceedings of the 25th USENIX Conference on Security Symposium, SEC’16*, page 327–343, USA, 2016. USENIX Association.
- [6] Sanjeev Arora and Rong Ge. New algorithms for learning in presence of errors. In Luca Aceto, Monika Henzinger, and Jiří Sgall, editors, *Automata, Languages and Programming*, pages 403–415, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.
- [7] Roberto Avanzi, Joppe Bos, Leo Lucas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehle. CRYSTALS-Kyber Algorithm Specifications And Supporting Documentation (version 3.02), 2021. <https://pq-crystals.org/kyber/data/kyber-specification-round3-20210804.pdf>.
- [8] Paul Barrett. Implementing the rivest shamir and adleman public key encryption algorithm on a standard digital signal processor. In Andrew M. Odlyzko, editor, *Advances in Cryptology — CRYPTO’ 86*, pages 311–323, Berlin, Heidelberg, 1987. Springer Berlin Heidelberg.
- [9] Mihir Bellare, Thomas Ristenpart, and Stefano Tessaro. Multi-instance security and its application to password-based cryptography. In Reihaneh Safavi-Naini and Ran Canetti, editors, *Advances in Cryptology - CRYPTO 2012 - 32nd Annual Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2012. Proceedings*, volume 7417 of *Lecture Notes in Computer Science*, pages 312–329. Springer, 2012.
- [10] Jonas Bertels, Quinten Norga, and Ingrid Verbauwhede. A better kyber butterfly for FPGAs. *Cryptology ePrint Archive*, Paper 2024/1367, 2024.
- [11] Joppe W. Bos, Craig Costello, Léo Lucas, Ilya Mironov, Michael Naehrig, Valeria Nikolaenko, Ananth Raghunathan, and Douglas Stebila. Frodo: Take off the Ring! Practical, Quantum-Secure Key Exchange from LWE. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24-28, 2016*, pages 1006–1018. ACM, 2016.
- [12] Joppe W. Bos, Léo Lucas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS - kyber: A cca-secure module-lattice-based KEM. In *2018 IEEE European Symposium on Security and Privacy, EuroS&P 2018, London, United Kingdom, April 24-26, 2018*, pages 353–367. IEEE, 2018.
- [13] Leon Botros, Matthias J. Kannwischer, and Peter Schwabe. Memory-efficient high-speed implementation of kyber on cortex-m4. In Johannes Buchmann, Abderrahmane Nitaj, and Tajjeeddine Rachidi, editors, *Progress in Cryptology – AFRICACRYPT 2019*, pages 209–228, Cham, 2019. Springer International Publishing.

- [14] Yuanmi Chen and Phong Q. Nguyen. Bkz 2.0: Better lattice security estimates. In Dong Hoon Lee and Xiaoyun Wang, editors, *Advances in Cryptology – ASIACRYPT 2011*, pages 1–20, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.
- [15] Dana Dachman-Soled, Léo Ducas, Huijing Gong, and Mélissa Rossi. LWE with side information: Attacks and concrete security estimation. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology - CRYPTO 2020 - 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17-21, 2020, Proceedings, Part II*, Lecture Notes in Computer Science, pages 329–358. Springer, 2020.
- [16] Léo Ducas, Alain Durmus, Tancrede Lepoint, and Vadim Lyubashevsky. Lattice signatures and bimodal gaussians. In Ran Canetti and Juan A. Garay, editors, *Advances in Cryptology - CRYPTO 2013 - 33rd Annual Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2013. Proceedings, Part I*, Lecture Notes in Computer Science, pages 40–56. Springer, 2013.
- [17] Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-dilithium: A lattice-based digital signature scheme. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2018(1):238–268, 2018.
- [18] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In Michael Wiener, editor, *Advances in Cryptology — CRYPTO’ 99*, pages 537–554, Berlin, Heidelberg, 1999. Springer Berlin Heidelberg.
- [19] Nicolas Gama, Phong Q. Nguyen, and Oded Regev. Lattice enumeration using extreme pruning. In Henri Gilbert, editor, *Advances in Cryptology – EUROCRYPT 2010*, pages 257–278, Berlin, Heidelberg, 2010. Springer Berlin Heidelberg.
- [20] Archisman Ghosh, Ming-Che Li, Lingke Ding, Suparna Kundu, Abdullah Sayeed, Angshuman Karmakar, Harshit Naman, Sarthak Antal, Ingrid Verbauwhede, and Shreyas Sen. 25.3 a 16nm 0.042mm² 0.66μ j/ops lightweight mlwe pqc kem with cryptanalysis-asic co-optimization. In *2026 IEEE International Solid-State Circuits Conference (ISSCC)*, volume 69, pages 428–430, 2026.
- [21] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A Modular Analysis of the Fujisaki-Okamoto Transformation. In Yael Kalai and Leonid Reyzin, editors, *Theory of Cryptography - 15th International Conference, TCC 2017, Baltimore, MD, USA, November 12-15, 2017, Proceedings, Part I*, volume 10677 of *Lecture Notes in Computer Science*, pages 341–371. Springer, 2017.
- [22] Matthias J. Kannwischer, Richard Petri, Joost Rijneveld, Peter Schwabe, and Ko Stoffelen. PQM4: Post-quantum crypto library for the ARM Cortex-M4. <https://github.com/mupq/pqm4>.
- [23] Angshuman Karmakar, Jose Maria Bermudo Mera, Sujoy Sinha Roy, and Ingrid Verbauwhede. Saber on ARM cca-secure module lattice-based key encapsulation on ARM. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2018(3):243–266, 2018.
- [24] Paul Kirchner and Pierre-Alain Fouque. An improved bkz algorithm for lwe with applications to cryptography and lattices. In Rosario Gennaro and Matthew Robshaw, editors, *Advances in Cryptology – CRYPTO 2015*, pages 43–62, Berlin, Heidelberg, 2015. Springer Berlin Heidelberg.
- [25] Suparna Kundu, Archisman Ghosh, Angshuman Karmakar, Shreyas Sen, and Ingrid Verbauwhede. Rudraksh: A compact and lightweight post-quantum key-encapsulation mechanism. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2025(2):647–680, 2025.
- [26] Xianhui Lu, Yamin Liu, Zhenfei Zhang, Dingding Jia, Haiyang Xue, Jingnan He, Bao Li, and Kunpeng Wang. LAC: Practical ring-LWE based public-key encryption with byte-level modulus. Cryptology ePrint Archive, Paper 2018/1009, 2018.
- [27] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In Henri Gilbert, editor, *Advances in Cryptology - EUROCRYPT 2010, 29th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Monaco / French Riviera, May 30 - June 3, 2010. Proceedings*, Lecture Notes in Computer Science, pages 1–23. Springer, 2010.

- [28] NIST. Submission Requirements and Evaluation Criteria for the Post-Quantum Cryptography Standardization Process, 2016. <https://csrc.nist.gov/CSRC/media/Projects/Post-Quantum-Cryptography/documents/call-for-proposals-final-dec-2016.pdf>.
- [29] Institute of Commercial Cryptography Standards. Guidelines for self-assessment of cryptographic algorithms performance on x86 architecture, 2026. Accessed: 2026-06-23.
- [30] Thales Paiva, Marcos Simplicio, Syed Hafiz, Bahattin Yildiz, Eduardo Cominetti, and Henrique Ogawa. Tailorable codes for lattice-based kems with applications to compact ml-kem instantiations. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025:139–163, 06 2025.
- [31] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *J. ACM*, 56(6), September 2009.
- [32] Chen Rui. Fpga sm3 hash, 2018. Accessed: 2026-06-29.
- [33] Tsunekazu Saito, Keita Xagawa, and Takashi Yamakawa. Tightly-secure key-encapsulation mechanism in the quantum random oracle model. *IACR Cryptol. ePrint Arch.*, 2017:1005, 2017.
- [34] Ardianto Satriawan, Rella Mareta, and Hanho Lee. A complete beginner guide to the number theoretic transform (NTT). Cryptology ePrint Archive, Paper 2024/585, 2024.
- [35] Claus Schnorr and M. Euchner. Lattice basis reduction: Improved practical algorithms and solving subset sum problems. *Mathematical Programming*, 66:181–199, 08 1994.